Blockchain-Enabled Efficient Deduplication and Mixed Auditing for Dynamic Cloud Data

Chunfei Pan, Lei Zhou, Anmin Fu, *Member, IEEE*, Zhenzhu Chen, *Member, IEEE*, Huaqun Wang, Yifeng Zheng, *Member, IEEE*, and Yansong Gao, *Senior Member, IEEE*

***Abstract*—As cloud storage is extensively utilized in the contemporary digital age, assuring data integrity and conserving cloud storage space has become a priority for all. However, existing cross-user deduplication audit schemes conflict with the pay-as-you-go model, causing unnecessary costs and violating data isolation. Moreover, retaining a single copy of identical data across multiple users introduces maintenance challenges during data operations. To address these issues, we propose a new blockchain-enabled efficient deduplication and mixed auditing scheme which intricately integrates Message-Locked Encryption (MLE) to construct Homomorphic Verifiable Tags (HVTs), enabling deduplication without exposing confidential data. Our scheme supports single-user deduplication at both block and file levels, as well as plaintext-ciphertext mixed auditing, thereby preventing redundant payments while preserving data isolation to simplify maintenance during data operations and ownership transfers. By employing Elliptic Curve Cryptography (ECC) to encrypt keys and storing the encrypted keys on the blockchain, we ensure data confidentiality while reducing the burden of local key management. Leveraging blockchain-based smart contracts, we further design a self-auditing mechanism that eliminates reliance on trusted third-party auditors. Moreover, our scheme embraces dynamic data operations through an optimized Merkle Hash Tree (MHT) and enables secure cloud data ownership transfer via identity verification. Finally, we prove the correctness and security of our scheme and evaluate its performance through experiments and comparisons with state-of-the-art works, demonstrating its efficiency, particularly in the data upload phase.**

***Index Terms*—Cloud storage, blockchain, deduplication, mixed auditing, data dynamics.**

Received 11 September 2025; revised 24 October 2025; accepted 23 November 2025. Date of publication 27 November 2025; date of current version 12 March 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62302330, Grant 62402397, Grant 62372236, and Grant U23B2002, in part by the Natural Science Foundation of the Higher Education Institutions of Jiangsu Province under Grant 23KJD520013, in part by the Natural Science Foundation of Jiangsu Province under Grant BK20230727, in part by the Youth Science and Technology Talent Support Project of Jiangsu Province under Grant JSTJ-2024-662, and in part by the Qinglan Project of Jiangsu Province of China 2025. *(Chunfei Pan and Lei Zhou are co-first authors.) (Corresponding author: Anmin Fu.)*

Chunfei Pan and Anmin Fu are with the School of Computer Science and Engineering, Nanjing University of Science and Technology, Nanjing 210094, China (e-mail: chunfei.pan@njust.edu.cn; fuam@njust.edu.cn).

Lei Zhou is with the School of Information Engineering, Suqian University, Suqian 223800, China (e-mail: zhoulei@squ.edu.cn).

Zhenzhu Chen is with the School of Computing and Artificial Intelligence, Southwestern University of Finance and Economics, Chengdu 611130, China (e-mail: chenzz@swufe.edu.cn).

Huaqun Wang is with the School of Computer Science, Nanjing University of Posts and Telecommunications, Nanjing 210023, China (e-mail: wanghuaqun@aliyun.com).

Yifeng Zheng is with the Department of Electrical and Electronic Engineering, The Hong Kong Polytechnic University, Hong Kong, SAR, China (e-mail: yifeng.zheng@polyu.edu.hk).

Yansong Gao is with the Department of Computer Science and Software Engineering, The University of Western Australia, Perth, WA 6009, Australia (e-mail: garrison.gao@uwa.edu.au).

Digital Object Identifier 10.1109/TDSC.2025.3638157

I. Introduction

In the era of Big Data, the volume of data is growing explosively. To alleviate the local storage burden, an increasing number of users are opting to outsource a significant proportion of data storage in order to effectively ease the local storage burden [1]. Cloud Service Providers (CSPs) play a pivotal role in this process by offering storage services that are typically priced according to the amount of data stored and the duration of service. Major CSPs in the current market include DropBox, OneDrive, Google Drive, and AWS. In practice, users select appropriate CSPs based on cost and the reliability of their storage services.

However, incidents of data loss and corruption continue to occur in cloud environments despite CSPs' claims of providing reliable services [2]. To preserve their reputation, CSPs may even resort to deceptive practices by falsely assuring users that their data remains intact on cloud servers [3]. To mitigate this risk, numerous integrity auditing schemes [4], [5], [6] have been proposed, enabling users to verify the correctness of data stored in the cloud. Most existing schemes, however, either impose significant auditing and verification overhead on users or rely on a trusted Third-Party Auditor (TPA) [7], limiting their practicality in real-world deployments.

A. Motivation

Current cloud storage systems suffer from substantial redundancy, with studies indicating that nearly 75% of stored data is duplicated [8], [9], [10]. To optimize storage utilization, existing research has proposed outsourced deduplication schemes [11], [12], which eliminate duplicate data uploaded by multiple users to reduce storage overhead. Common data deduplication schemes primarily include file-level deduplication and block-level deduplication [13]. File-level deduplication compares the entire content of a data file to determine whether an identical copy already exists on the cloud server, retaining only one copy of the original file. In contrast, block-level deduplication employs a finer granularity, typically dividing the data file into multiple blocks and comparing the contents of each block to identify duplicates. Nevertheless, these designs face two key limitations. (1) Current deduplication schemes, designed to achieve

1545-5971 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on March 29,2026 at 13:35:55 UTC from IEEE Xplore. Restrictions apply.

cross-user file deduplication, are fundamentally incompatible with the prevailing "pay-as-you-go" cloud storage model, which charges users based on the amount of data stored and its retention duration. In these schemes, multiple users may incur duplicate payments. This issue arises because current cross-user deduplication allows data to be shared among users, resulting in each user having to pay for the same storage copy. Furthermore, this approach violates the principle of user data isolation, as the same physical data may be accessed or shared by multiple users. This situation not only renders financial problems but also privacy and security risks. (2) The deduplication designs do not take into account data dynamics and data ownership transactions over the stored data. This is because keeping only one copy of the identical data leads to ambiguity in determining data ownership beneficiaries during buy-sell transactions. Moreover, with only one copy remaining, ongoing data maintenance will encounter difficulties, since different users may process the same data diversely.

To overcome the aforementioned limitations, this work proposes a scheme that enables single-user data deduplication at both the file and block level, while simultaneously ensuring dynamic data integrity and enabling seamless ownership transfers. By performing deduplication on the client side rather than on the cloud, the scheme substantially reduces redundant transmission overhead and eliminates the computational cost of generating duplicate authenticators. Additionally, it proposes a novel data integrity auditing pattern—plaintext-ciphertext mixed auditing. The design allows the outsourced data to simultaneously contain plaintext for public data and ciphertext for private data, supporting the integrity auditing of mixed plaintext and ciphertext data. This maintains the mixed plaintext-ciphertext structure, enabling users to selectively encrypt sensitive data segments. Compared to full-file encryption, this approach improves retrieval efficiency and computational performance. To the best of our knowledge, existing deduplication schemes predominantly lack support for both dynamic data operations and ownership transfers. In contrast, the proposed scheme maintains distinct user-specific copies of identical data, which offers significant advantages for handling data dynamics and ownership transactions, aligning more closely with practical market demands.

B. Contributions

Our contributions are as follows:

- This paper presents a novel HVT architecture tightly integrated with MLE, which enables both block-level and file-level deduplication for individual users while supporting mixed plaintext-ciphertext auditing. We employ ECC to efficiently encrypt data sector keys, thereby streamlining key management overhead. Moreover, the approach where multiple users each maintain one cloud copy of the same data enables subsequent individual data maintenance and data ownership transactions.
- Leveraging smart contracts, we design an automated auditing mechanism that eliminates reliance on TPA and implement a protocol to enable secure ownership transfers between different users. Through an enhanced MHT, the scheme further supports verifiable dynamic data updates. Additionally, we incorporate identity verification to prevent unauthorized data access and modifications.
- We provide rigorous security proofs for the proposed scheme, establishing its unforgeability, confidentiality, identity-based authentication security, and ownership transfer security. Theoretical analysis and experiments demonstrate efficient auditing performance with reduced tag generation overhead compared to existing schemes, particularly for files with limited confidential data.

C. Outline of This Paper

The paper is organized as follows. Section II reviews related auditing schemes. In Section III, we present definitions and preliminaries. Section IV details our scheme's design. Section V proves the scheme's correctness and security through provable security theory. Section VI evaluates performance experimentally. Finally, we conclude in Section VII.

II. Related Work

A. Data Integrity Auditing

Ateniese et al. [14] introduced Provable Data Possession (PDP), which verifies data integrity by randomly sampling data blocks stored on the server, thereby avoiding full data retrieval. Although validation requires only a small subset of blocks, the original PDP supports only static data. Subsequent research extended PDP to accommodate dynamic operations. Wang et al. [15] combined homomorphic linear authenticators with the MHT to support dynamic data auditing. Liu et al. [16] subsequently proposed an authorized public auditing framework using MHT to accommodate variable-sized block updates. Later, Yan et al. [17] further optimized data dynamics by introducing an operation record table to reduce update overhead. However, the aforementioned single-copy auditing schemes are inherently vulnerable to a single point of failure, which compromises data availability and reliability. Yu et al. [18] introduced a multi-copy auditing scheme that reduces update overhead through the use of indexed MHT. Nevertheless, storing multiple replicas on a single server remains vulnerable to server failures. To address this limitation, Li et al. [19] designed an identity-based PDP scheme for multi-cloud storage that enables simultaneous integrity verification across all replicas. In a similar vein, Mishra et al. [20] proposed a multi-cloud mechanism for electronic medical records, which ensures data recoverability while providing a low-cost auditing solution.

Data privacy and security are also critical concerns in audit schemes. Li et al. [21] randomized the challenged data blocks to ensure that the TPA cannot obtain any private information during the audit process. Nevertheless, identity-based privacy protection schemes suffer from the key escrow problem. To address this issue, Li et al. [22] further designed a certificateless signature-based audit scheme, which eliminates the need for certificate management and avoids the problem of key escrow. To further protect data privacy, Fu et al. [23] established a privacy-aware public auditing system for shared data through homomorphic

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on March 29,2026 at 13:35:55 UTC from IEEE Xplore. Restrictions apply.

verifiable group signatures, though they did not address the risk of exposing user identity information. Meanwhile, Gai et al. [24] employed random masking techniques to simultaneously protect data content and owner anonymity. Recently, Wang et al. [25] developed a time-constrained auditing protocol that limits TPA access to sensitive data, thereby enhancing data privacy and security.

B. Blockchain-Based Data Integrity Auditing

Integrity auditing schemes, including those previously discussed, rely on TPAs [26]. While TPA-based approaches effectively offload computationally intensive auditing tasks from users, real-world implementation faces challenges trusting these third parties. Yang et al. [27] innovatively employed mutual auditing among multiple servers in multi-cloud storage environments to eliminate third-party entities. However, the scheme fails to consider potential collusion among cloud servers to forge integrity proofs. The aforementioned limitations have motivated the adoption of blockchain technology, utilizing its transparency and immutability to transform auditing frameworks. Recent advances in blockchain auditing show promising developments, as Lin et al. [28] employed consortium blockchain smart contracts for automated public auditing while preventing auditor and cloud server collusion. Subsequently, Zhang et al. [29] also utilized smart contracts to interact with storage service providers for integrity verification, implementing distributed auditing through HVT and polynomial commitments. Similarly, Miao et al. [30] leveraged the decentralized nature of blockchain, ingeniously using a consensus mechanism combined with identity-based broadcast encryption to achieve integrity auditing, though this approach incurs significant communication overhead. To reduce the overhead of the scheme, Yu et al. [31] designed a dual-compression auditing scheme using on-chain and off-chain blockchain structures, significantly minimizing storage overhead. However, disputes may arise between users and CSPs regarding the audit results. To address this issue, Wang et al. [32] developed an arbitration scheme based on blockchain smart contracts. Recently, Zhang et al. [33] advanced certificate-less auditing based on smart contracts by integrating Lagrange interpolation with multi-signature HVT.

C. Deduplication and Data Integrity Auditing

Standalone auditing schemes are unable to adequately support modern storage models, primarily due to inefficient resource utilization caused by redundant cloud data storage. This limitation has prompted the development of integrated solutions that combine deduplication and auditing capabilities. Xu et al. [36] pioneered a blockchain-based approach that tracks entity behavior and supervises untrusted TPAs, while Tian et al. [35] advanced this field with a secure deduplication and shared auditing framework featuring lightweight authenticators and a two-server model that eliminates third-party auditors, though it overlooked data update requirements. Mishra et al. [37] innovatively introduced editable blockchain technology to replace TPAs for implementing block-level deduplication auditing. Subsequent efforts include Song et al.'s [38] encrypted storage solution distinguishing between initial and subsequent uploaders, Peng et al.'s [39] security-enhanced approach using dynamic auditing-though these still depend on TPAs. Afterwards, Zhang et al. [9] demonstrated a public auditing and deduplication scheme based on a double-copy storage model for blockchain off-chain storage, which can automatically realize user ownership authentication and integrity auditing. The proposed model necessitates double storage allocation for each individual data file. Zhang et al. [10] also achieved cross-user deduplication functionality. While their introduction of identity-based broadcast encryption reduces key management costs, it incurs additional overhead when transmitting encrypted key metadata. Lately, Liu et al. [34] proposed a scheme enabling internal ciphertext/tag deduplication and cross-user deduplication, though this approach necessitates multiple rounds of interaction between CSPs and users.

We compare our scheme with state-of-the-art approaches supporting both deduplication and auditing in Table I. The schemes [9], [10], [34], [35] are representative solutions, which not only support data integrity auditing but also implement data deduplication, all while utilizing blockchain technology. Therefore, we chose these schemes for a functional comparison with our approach. All schemes ensure data privacy; however, only our scheme adopts mixed plaintext-ciphertext encryption to enhance efficiency. With respect to auditing, our scheme is the only one that enables mixed auditing across both plaintext and ciphertext data. This auditing method allows for audits to be conducted even when users only encrypt a portion of the private data, thereby significantly improving the efficiency of the data encryption phase. All schemes remove reliance on TPAs, though Liu et al.'s scheme [34] still requires a trusted third party for signature generation. Our scheme, along with [9], [34], [35], supports block-level deduplication, whereas [10] is limited to file-level deduplication. In particular, our scheme performs deduplication on the user side. Most notably, only our scheme simultaneously supports data dynamics and ownership transfers, thereby offering superior practicality and efficiency.

III. Definitions and Preliminaries

A. System Model

As summarized in Table II, we give notations along with their detailed descriptions in the paper. Fig. 1 illustrates the system model, which is composed of three entities:

- User (U): An entity with limited local storage capacity. U processes and uploads mixed plaintext-ciphertext data files to the CSP for outsourced storage. In this arrangement, U is charged for the storage service as well as for audit operations. When performing dynamic updates, U submits a request to the CSP for the specified update.
- CSP: An entity with substantial storage capacity and computational resources. The CSP provides storage services to U under a pay-as-you-go model, while jointly performing integrity proof generation and file deduplication. However, to preserve its business reputation, the CSP may deliberately misrepresent the actual integrity status of data to U by issuing forged verification proofs.
- Blockchain: The distributed ledger system, characterized by immutability, is leveraged to reliably record and

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on March 29,2026 at 13:35:55 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARISONS OF FUNCTIONALITY

Scheme	Data confidentiality	No trusted entities	Mixed audit	Data deduplication	Data dynamic	Data ownership transactions
Zhang *et al.* [10]	✓	✓	✕	File-level	✕	✕
Liu *et al.* [34]	✓	✕	✕	Block-level	✕	✕
Zhang *et al.* [9]	✓	✓	✕	Block-level	✕	✕
Tian *et al.* [35]	✓	✓	✕	Block-level	✕	✕
Our scheme	✓	✓	✓	Block-level	✓	✓

✓: supported; ✕: not supported.

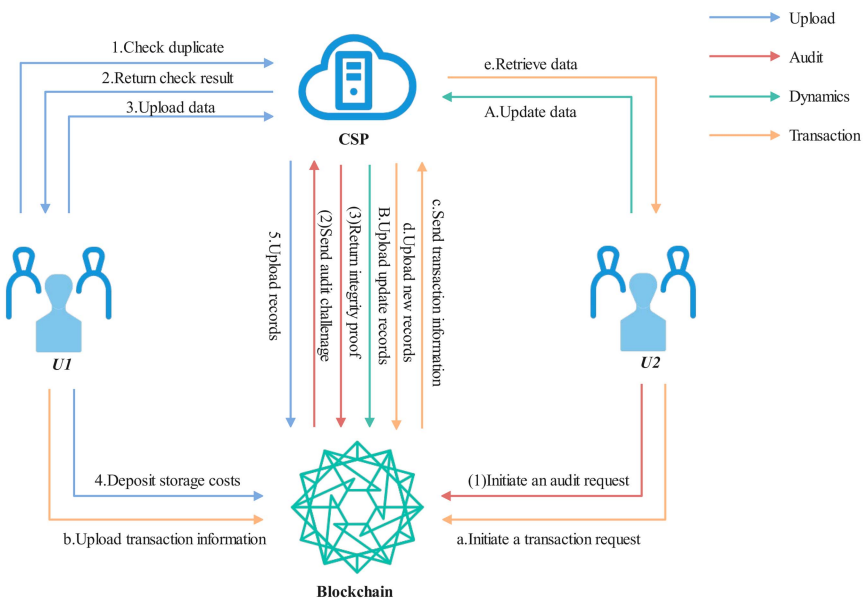


Fig. 1. The system model and workflow of the proposed scheme.

maintain storage metadata, audit logs, and ownership transaction records. By integrating smart contracts, the system autonomously performs integrity verification and simultaneously acts as an impartial arbitrator, governing the distribution of user-submitted service fees and CSP-provided deposits.

B. Definitions

Our scheme is based on a variant work of references [13], [35] and consists of the following collection of algorithms {Setup, KeyGen, Encrypt, TagGen, AuthGen, Decrypt, ProofGen, Modify, Insert, Delete} and a smart contract {AuditReq,VerifyProof} as follows:

- Setup $(\lambda) \rightarrow para$: Given a security parameter λ as input, the setup algorithm outputs a set of public parameters $para$.
- KeyGen: On input the data file $F = \{m_{i,j}\}_{1 \le i \le n, 1 \le j \le s}$, with being first divided into blocks and then further partitioned into sectors, the key generation algorithm outputs file key fk and sector key collection $\{k_{i,j}\}$:
  - F-KeyGen $(F) \rightarrow fk$: Uses the file F as input, outputs the file key fk.
  - S-KeyGen $(m_{i,j}) \rightarrow k_{i,j}$: Uses the data sector $m_{i,j}$ as input, outputs the sector key $k_{i,j}$.
- Encrypt: On input the confidential data sectors $\{m_{i_c,j}\}$ and the sector key collection $\{k_{i,j}\}$ and the ECC public key pk, the encrypt algorithm outputs the sector ciphertext $\{c_{i_c,j}\}$ and the ciphertext of sector keys $\{Key_i\}$.
  - S-Encrypt $(k_{i_c,j}, m_{i_c,j}) \rightarrow c_{i_c,j}$: Uses confidential data sector $m_{i_c,j}$ and the corresponding sector key $k_{i_c,j}$ as input, outputs the sector ciphertext $c_{i_c,j}$.
  - K-Encrypt $(pk, \{k_{i,j}\}) \rightarrow Key_i$: Uses the sector keys $\{k_{i,j}\}$ for i-th block and the ECC public key pk as input, outputs the ciphertext of sector keys Key_i.
- TagGen $(c_i) \rightarrow tg_i$: The tag generation algorithm outputs tags $\{tg_i\}$ by taking the processed blocks $\{c_i\}$ as input.
- AuthGen $(\{c_{i,j}\}, tg_i, y_i) \rightarrow \sigma_i$: The authenticator generation algorithm outputs authenticators $\{\sigma_i\}$ by taking the sectors $\{c_{i,j}\}_{1 \le j \le s}$, tags $\{tg_i\}$ and privacy parameters $\{y_i\}$ as input.
- Decrypt: On input the sector ciphertext $\{c_{i_c,j}\}$ and the ciphertext of sector keys $\{Key_i\}$ and the ECC private key sk as input, the decryption algorithm outputs the decrypted data sectors $\{m_{i_c,j}\}$.
  - K-Decrypt $(sk, Key_i) \rightarrow \{k_{i,j}\}$: Uses the ciphertext of sector keys Key_i for i-th block and the ECC private key sk as input, outputs the sector key collection $\{k_{i,j}\}$.
  - S-Decrypt $(k_{i_c,j}, c_{i_c,j}) \rightarrow m_{i_c,j}$: Uses the encrypted sector $c_{i_c,j}$ and the corresponding sector key $k_{i_c,j}$ as input, output the sector plaintext $m_{i_c,j}$.
- AuditReq $(req) \rightarrow Chal$: The algorithm takes the audit request req as input and outputs the audit challenge $Chal$.

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on March 29,2026 at 13:35:55 UTC from IEEE Xplore. Restrictions apply.

TABLE II
NOTATIONS AND DESCRIPTIONS

Notation	Description
F	Plaintext of the data file to be uploaded
F_1	Publicly available data segment of F
F_2	Confidential data segment of F
I_1	Set of data block serial numbers in F_1
I_2	Set of data block serial numbers in F_2
Q	Set of the challenged data block serial numbers
m_i	The i-th data block
m_{i_p}	The i-th publicly available data block
m_{i_c}	The i-th confidential data block
c_i	The i-th ciphertext data block
c_{i_c}	The i-th confidential ciphertext data block
$m_{i,j}$	The j-th sector of the i_p-th data block
$m_{i_p,j}$	The j-th sector of the i-th block of publicly available data
$m_{i_c,j}$	The j-th sector of the i-th block of confidential data
$c_{i,j}$	The j-th sector of the i-th ciphertext data block
$c_{i_c,j}$	The j-th sector of the i-th block of confidential ciphertext data
$\mathbb{C}$	Processed data set
fk	Key of data file F
$k_{i,j}$	Key of data sector $m_{i,j}$
$k_{i_c,j}$	Key of confidential data sector $m_{i_c,j}$
Key_i	Ciphertext of the key of data sector
n	Number of data blocks
s	Number of sectors into which each block is divided
ζ	Number of publicly available data blocks in F
d	Number of deduplicate data blocks in F
$\mathbb{G}_1, \mathbb{G}_2$	Two multiplicative cyclic groups
Z_p^*	Residue class ring
H_1, H_2, H_3, H_4	Four hash functions
f_1	A pseudo-random permutation
f_2	A pseudo-random function
e	A bilinear map

- ProofGen $(Chal, \{c_i\}) \rightarrow P$: On input the audit challenge $Chal$ and the challenged data blocks $\{c_i\}_{i \in Q}$, the proof generation algorithm outputs the integrity proof P.
- VerifyProof $(P) \rightarrow 0/1$: The algorithm takes the integrity proof P as input, and outputs the audit result $0/1$, where 1 corresponds to successful verification, and 0 indicates verification failure.
- Modify $(Modify, t, i, tg_i^*, c_i^*) \rightarrow w_r^*$: On input the original data tag t, the number i of the data block being modified, the new tag tg_i^* of the new data block, and the new data block c_i^*, the modify algorithm outputs the root node w_r^* of the updated MHT.
- Insert $(Insert, t, i, tg_{i+1}^*, c_{i+1}^*) \rightarrow w_r^*$: On input the data tag t, the number i of the previous data block at the insertion location, the tag tg_{i+1}^* of the newly inserted data block and the new data block c_{i+1}^*, the insert algorithm outputs the root node w_r^* of the updated MHT.
- Delete $(Delete, t, i) \rightarrow w_r^*$: On input the data tag t and the number i of deleted data block, the delete algorithm outputs the root node w_r^* of the updated MHT.

C. Threat Model

We primarily consider threats from two semi-trusted entities:

- *Semi-Trusted CSP:* During data storage, the CSP may lose data due to hardware failures or other issues. To protect its business reputation, it may falsely claim that the data remains intact in the cloud. In integrity audits, it may also forge authenticators or audit evidence to deceive the verifier.
- *Semi-Trusted User* (Original Data Owner in Ownership Transfer): In data ownership transactions, the original owner may provide a fake decryption key to the buyer. After the transfer, the original owner may further attempt to illegally access the data despite no longer retaining ownership rights.

We assume that none of the aforementioned parties will engage in actions detrimental to their own interests. Meanwhile, we assume that the blockchain is absolutely secure, capable of reliably recording transaction information, and that smart contracts can be executed correctly.

D. Security Model

In our proposed scheme, we mainly consider the adversary $\mathcal{A}$ (simulating a semi-honest CSP) who tries to pass the audit by querying the data block information and thus forging the authenticators and integrity proofs. To formalize our security framework, we introduce two games between a challenger $\mathcal{C}$ and the adversary $\mathcal{A}$.

1) Unforgeability of Authenticator: Game 0. The game is run between $\mathcal{C}$ and $\mathcal{A}$.

- *Setup*. $\mathcal{C}$ runs the Setup algorithm to generate the public parameters $para$. Then $\mathcal{C}$ sends $para$ to $\mathcal{A}$.
- *Query*. $\mathcal{A}$ adaptively performs H_1-H_4 hash queries and authenticator queries, and $\mathcal{C}$ returns the corresponding values to $\mathcal{A}$.
- *Forge*. $\mathcal{A}$ outputs a forged authenticator σ_c'. If the forged authenticator can pass the verification with non-negligible probability as follows, $\mathcal{A}$ wins, otherwise $\mathcal{A}$ fails. We define the adversary $\mathcal{A}$'s advantage in *Game 0* to be

$$Adv_{\mathcal{A}}^{Game0} = Pr_{\mathcal{A}}[\sigma_c' = \sigma_c].$$

Definition 1: The scheme is considered to satisfy the unforgeability of authenticators if for any Probabilistic Polynomial Time (PPT) adversary $\mathcal{A}$, its advantage in the security game is negligible:

$$Adv_{\mathcal{A}}^{Game0} \leq negl(\lambda),$$

where λ is a security parameter, and the probability is taken over the coin tosses of both $\mathcal{C}$ and $\mathcal{A}$.

2) Unforgeability of Proof: Game 1. The game is run between $\mathcal{C}$ and $\mathcal{A}$. The *Setup* and *Query* phases are identical to those in *Game 0*, and we omit them here.

- *Challenge*. $\mathcal{C}$ generates a random challenge $Chal$ indicating some of the unqueried data blocks. Based on $Chal$, $\mathcal{A}$ computes the corresponding integrity proof P and returns them to $\mathcal{C}$.
- *Verify*. $\mathcal{C}$ verifies the integrity proof P'. If P' can pass the verification with non-negligible probability as follows, $\mathcal{A}$ wins, otherwise $\mathcal{A}$ fails. We define the adversary $\mathcal{A}$'s advantage in *Game 1* to be

$$Adv_{\mathcal{A}}^{Game1} = Pr_{\mathcal{A}}[VerifyProof(P') = 1].$$

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on March 29,2026 at 13:35:55 UTC from IEEE Xplore. Restrictions apply.

Definition 2: The scheme is considered to satisfy the unforgeability of proof if for any PPT adversary $\mathcal{A}$, its advantage in the security game is negligible:

$$Adv_{\mathcal{A}}^{Game1} \leq negl(\lambda),$$

where λ is a security parameter, and the probability is taken over the coin tosses of both $\mathcal{C}$ and $\mathcal{A}$.

E. Design Objectives

Our proposal is designed to achieve the following objectives:

- **Automatic auditing**: The scheme leverages smart contracts to automatically perform audits, verifying the CSP's integrity proofs and ensuring audit correctness.
- **User-side file/block deduplication**: The proposal supports deduplication at both file and block levels on the user side, thereby reducing storage and communication costs associated with duplicate data and authenticators.
- **Mixed plaintext-ciphertext auditing**: The scheme enables auditing over mixed plaintext and ciphertext data, offering higher computational efficiency compared to full-file encryption approaches.
- **Data dynamics**: The scheme fully supports dynamic block-level operations, including modification, insertion, and deletion.
- **Data ownership transfer**: Through smart contracts, our scheme enables secure transfer of cloud data ownership in exchange for payment.
- **Defense against unauthorized access**: The scheme incorporates user identity verification to prevent unauthorized access and malicious data modifications.

IV. The Proposed Scheme

A. Overview

This paper presents a novel scheme that supports both user-side deduplication and mixed plaintext-ciphertext verification. Fig. 1 illustrates the system architecture and operational workflow, where the primary interactions are as follows: (1) Prior to data outsourcing, U queries the CSP to verify whether duplicates of the data already exist. Based on the CSP's response, U uploads only unique data to the CSP and deposits the corresponding storage fees on the blockchain, while the CSP records storage metadata and deposits its margin on the blockchain. (2) During auditing, U submits an audit request to the smart contract, which automatically triggers an integrity verification challenge. (3) Upon receiving the challenge, the CSP generates the corresponding integrity proof, which is verified by the smart contract, with the final audit result being immutably recorded on the blockchain. (4) For data updates, U submits an update request to the CSP, which executes the update operation and commits the corresponding verification evidence to the blockchain. (5) For ownership transfers between different users (e.g., $U1 \rightarrow U2$ in Fig. 1), $U1$ and $U2$ finalize the transaction through the smart contract, thereby ensuring secure and verifiable transfer of data ownership.

B. System Design

In our scheme, U, CSP and blockchain will interact as follows:

1) **Setup**: The system chooses a security parameter λ and executes the setup algorithm Setup (λ). Two multiplicative cyclic groups $\mathbb{G}_1$ and $\mathbb{G}_2$ of order p are chosen to construct a computable bilinear map $e : \mathbb{G}_1 \times \mathbb{G}_1 \rightarrow \mathbb{G}_2$, where p is a large prime number. The system selects a generator $g \in \mathbb{G}_1$, random elements $R = \{r_j\}_{1 \leq j \leq s} \in \mathbb{G}_1$, four hash functions $H_1 : \{0,1\}^* \rightarrow Z_p^*$, $H_2 : \{0,1\}^* \rightarrow \mathbb{G}_1$, $H_3 : Z_p^* \rightarrow Z_p^*$, $H_4 : \{0,1\}^l \rightarrow \mathbb{G}_1$, a pseudo-random permutation $f_1 : \{1,\ldots,n\} \times Z_p^* \rightarrow \{1,\ldots,n\}$ and a pseudo-random function $f_2 : \{1,\ldots,n\} \times Z_p^* \rightarrow Z_p^*$. Finally, the system publishes the public parameters $para = \{\mathbb{G}_1, \mathbb{G}_2, p, e, g, R, H_1, H_2, H_3, H_4, f_1, f_2\}$.

2) **Data Upload and Deduplication**: Prior to file upload, U divides the file F into n equal-sized data blocks of length $s|p|$, expressed as $F = F_1 \cup F_2 = \{m_i\}_{1 \leq i \leq n}$, where $F_1 = \{m_{i_p}\}_{i_p \in I_1}$ constitutes the ζ publicly available data blocks and $F_2 = \{m_{i_c}\}_{i_c \in I_2}$ represents the confidential data blocks requiring encryption, with I_1 and I_2 respectively denoting the sets of public and confidential block indices in F; when encountering a final block shorter than $s|p|$, U pads its end with zeros, then further partitions each block into s sectors from Z_p^*, yielding the final file representation $F = \{m_{i,j}\}_{1 \leq i \leq n, 1 \leq j \leq s}$, upon which U proceeds to compute the keys and tags through subsequent steps.

- *Key Generation:* U invokes KeyGen algorithm to generate the file key and sector keys. U uses the hash value of the file to compute the file key $fk = H_1(F)$. Similarly, U generates the sector key $k_{i,j} = H_1(m_{i,j})$ based on the data sectors $\{m_{i,j}\}_{1 \leq i \leq n, 1 \leq j \leq s}$.
- *Tag Generation:* U calculates the tags for the file and its divided data blocks. For encrypting confidential blocks $F_2 = \{m_{i_c}\}_{i_c \in I_2}$, U invokes $\mathsf{Encrypt}$ algorithm to get the block ciphertext $\{c_{i_c}\}$, where each sector $c_{i_c,j}$ of c_{i_c} is computed by $c_{i_c,j} = H_3(k_{i_c,j}) \oplus m_{i_c,j}$. For public data blocks $F_1 = \{m_{i_p}\}_{i_p \in I_1}$, U does not perform any processing. Eventually, U obtains a collection of processed data blocks as follows:

$$\mathbb{C} = \{c_i\}_{1 \leq i \leq n} = \begin{cases} m_{i_p}, i_p \in I_1 \\ c_{i_c}, i_c \in I_2 \end{cases}. \tag{1}$$

Afterwards, U invokes TagGen to calculate the block tags. Specifically, the file tag t is computed using the file key fk, and the block tags tg_i are hashed from data blocks as follows:

$$\begin{cases} t = g^{fk}, \\ tg_i = H_1(c_i). \end{cases} \tag{2}$$

To safeguard against private data leakage from key exposure and reduce the complexity of local key management, we utilize the ECC algorithm to encrypt the sector keys. Specifically, U selects a generator G on the `Secp256k1` elliptic curve, randomly generates a private key $sk \in \mathbb{Z}_p^*$, and computes the corresponding public key as $pk = sk \cdot G$. The sector key is then encrypted to produce the ciphertext $Key_i = Encrypt(pk, k_{i,1} \parallel \cdots \parallel k_{i,j})$. This approach allows U to retain only the ECC private key sk,

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on March 29,2026 at 13:35:55 UTC from IEEE Xplore. Restrictions apply.

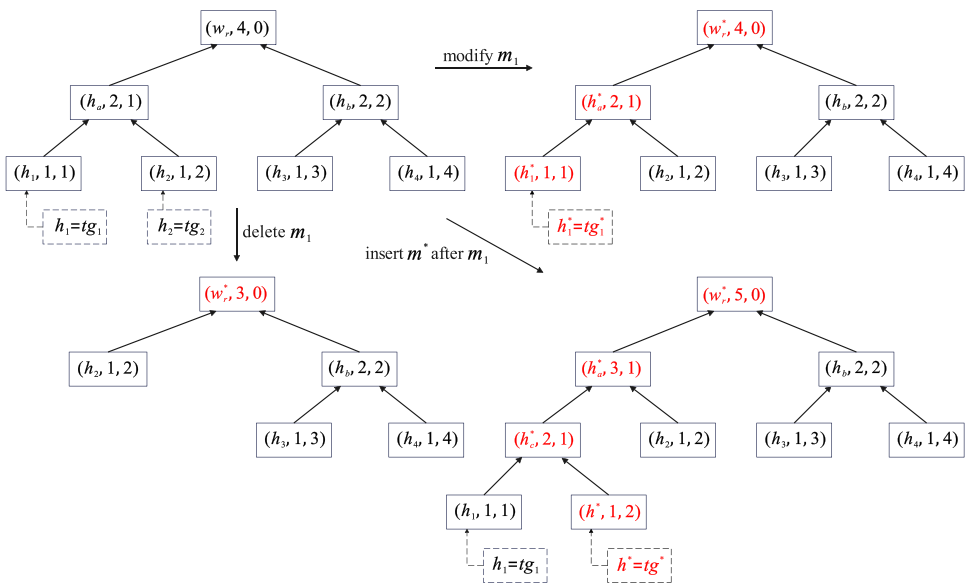


Fig. 2. Construction of the improved MHT structure.

significantly reducing local storage overhead while maintaining security unlike the alternative of storing all block keys individually. Additionally, U selects a secret random number $\mu \in \mathbb{Z}_p^*$ and derives the user identity components $UID = (H_4(uid) \cdot t)^\mu$ and $W = g^\mu$, where $uid \in \{0,1\}^l$ serves as U's unique identifier.

Through the above steps, U obtains the uploaded data set $\mathbb{C}$, file tag t, data block tag set $T = \{tg_i\}$, identity information $\{UID, W\}$, and key ciphertext $\{Key_i\}$.

- *Duplicate Checking:* U selects a suitable CSP for data storage. Initially, U transmits the deduplication verification tuple $\langle t, T, \{Key_i\}, uid, UID, W\rangle$ to the CSP via an off-chain communication channel. The CSP then authenticates U's identity information to prevent unauthorized data access, verifying the legitimacy through the following equation:

$$e(UID, g) \stackrel{?}{=} e(H_4(uid) \cdot t, W). \tag{3}$$

After successfully verifying U's identity, the CSP first performs a file-level deduplication check by querying its storage system using the file tag t. If an identical file is found to exist, the CSP immediately terminates the storage process. Otherwise, it proceeds to conduct block-level deduplication according to T to find the unique data block tag set T_u and returns T_u to U. Subsequently, U randomly selects $\gamma \in Z_p^*$ to compute $y_i = (\sum_{j=1}^{s} k_{i,j}) \oplus \gamma$ for each unique data block. U further calculates the public value $Y_i = g^{y_i}$ and invokes the AuthGen algorithm to generate the authenticator σ_i for the unique block:

$$\sigma_i = \left(H_2(s \| tg_i) \cdot \prod_{j=1}^{s} r_j^{c_{i,j}} \right)^{y_i}. \tag{4}$$

U uploads the set $\langle \mathbb{C}_u, \sigma = \{\sigma_i\}\rangle$ of the unique blocks $\mathbb{C}_u$ and corresponding authenticators. According to the size of all unique data blocks and the storage duration, U deposits the corresponding storage cost fee_1 to the blockchain smart contract. After receiving the upload information, CSP verifies the consistency of the uploaded data blocks and corresponding authenticators through the (5). If it passes the verification, then CSP stores the tuple $\langle t, T, \{Key_i\}, T_u, \mathbb{C}_u\rangle$. If the verification fails, the procedure is terminated.

$$e(\sigma_i, g) \stackrel{?}{=} e\left(H_2(s \| tg_i) \cdot \prod_{j=1}^{s} r_j^{c_{i,j}}, Y_i \right). \tag{5}$$

After completing data storage, CSP constructs an enhanced MHT structure where each node contains a triple (h, l_N, p). For leaf nodes, the h value represents the block tag, specifically $h_i = tg_i$, while for internal nodes, h is computed by pairwise aggregation of child nodes as $h = H_1(h_{lchildren} \| h_{rchildren})$. The l_N parameter denotes the count of accessible leaf nodes, calculated as $l_N = l_{lchildren} + l_{rchildren}$, and p indicates the node's positional index within its layer, with the root node's position fixed as $p = 0$. Finally, the root w_r is derived as shown in Fig. 2. Finally, CSP deposits the storage margin $Fee_1 = 2fee_1$ and uploads the storage information $Info = \langle t, T_u, \sigma, uid, SID, w_r\rangle$ to the blockchain smart contract, where SID is the CSP's identifier. We stipulate that if CSP does not store the data correctly, U will be compensated 2 times the cost of storage. U deletes the local file, retaining only the tuple $\langle t, sk\rangle$ for subsequent access.

3) **Mixed Auditing**: To verify the integrity of the outsourced data, U can invoke AuditReq algorithm to initiate a mixed audit request req and deposit an audit fee fee_2.

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on March 29,2026 at 13:35:55 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Audit Proof Generation.

Input: $Chal = (z, \theta_1, \theta_2)$
Output: P
Set $P \leftarrow \emptyset$
for $j = 1; j \leq s; j++$ **do**
  Set $P_j \leftarrow 0$
  for $i = 1; i \leq z; i++$ **do**
    Compute $x_i = f_1(i, \theta_1)$
    Compute $v_i = f_2(i, \theta_2)$
    if is stored c_{x_i} **then**
      Compute $P_j \leftarrow P_j + c_{x_i,j} v_i$
    end if
  end for
  $P \leftarrow P \cup \{P_j\}$
end for

- *Challenge Generation:* Upon receiving an audit request from the user, the smart contract automatically generates a challenge tuple $Chal = (z, \theta_1, \theta_2)$, where z is random index within the range $[1, n]$, and θ_1, θ_2 are randomly selected elements from Z_p^*. Finally, the smart contract publishes the challenge $Chal$ into the blockchain.
- *Proof Generation:* CSP obtains the audit challenge $Chal$ from the blockchain and invokes ProofGen algorithm, refer to Algorithm 1, where the set of x_i is represented as Q, to generate the integrity proof $P = \{P_j\}_{1 \leq j \leq s}$ based on the queried data block as follows:

$$P_j = \sum_{i=1}^{z} v_i c_{i,j}, \tag{6}$$

where $c_{i,j}$ is the sectors of the challenged blocks. Eventually, CSP publishes the integrity proof P into the blockchain.

Algorithm 2: Audit Proof Verification.

Input: P
Output: $result = 1/0$
Set $result \leftarrow 0$ //$non-integral$
for $i = 1; i \leq z; i++$ **do**
  Compute $x_i = f_1(i, \theta_1)$
  Compute $v_i = f_2(i, \theta_2)$
end for
if $e(\sigma_c, g) = e(\prod_{i=1}^{z}(H_2(s\|tg_{x_i})^{v_i} \cdot \prod_{j=1}^{s} r_j^{P_j}), \prod_{i=1}^{z} Y_{x_i})$ **then**
  $result \leftarrow 1$ //$integral$
end if

- *Proof Verification:* Upon receiving the integrity proof, the smart contract computes the aggregation authenticator σ_c as follows:

$$\sigma_c = \prod_{i=1}^{z} \sigma_{x_i}^{v_i}. \tag{7}$$

Then, the smart contract automatically executes VerifyProof algorithm to validate the proof's authenticity. The complete procedure is specified in Algorithm 2. The verification succeeds if (8) is satisfied, confirming that the CSP has properly maintained the outsourced data and thereby passes the audit. Conversely, it proves that the data has been corrupted or lost.

$$e(\sigma_c, g) \stackrel{?}{=} e\left(\prod_{i=1}^{z} H_2(s\|tg_{x_i})^{v_i} \cdot \prod_{j=1}^{s} r_j^{P_j}, \prod_{i=1}^{z} Y_{x_i}\right). \tag{8}$$

Ultimately, the smart contract records the audit outcome by publishing the tuple $\langle \sigma_c, P, result \rangle$ on the blockchain. A result value $result = 1$ indicates successful audit verification. After the specified storage period, if $\prod result = 1$, the smart contract transfers U's prepaid audit fee fee_2 to CSP. Instead, the smart contract transfers the margin Fee_1 to U.

4) **Data Retrieval**: U initiates a data retrieval request by sending the tuple $\langle t, UID, W \rangle$ to the CSP. The CSP first authenticates U's identity by verifying (3). Upon successful authentication, CSP returns the corresponding data and key ciphertext $\langle \mathbb{C}, \{Key_i\} \rangle$ to U. U then proceeds with data recovery in two phases: directly accessing available plaintext portions, and for encrypted content, decrypting the sector keys $\{k_{i,j}\} = Decrypt(sk, Key_i)$ using the locally stored private key sk, followed by computing $m_{i_c,j} = H_3(k_{i_c,j}) \oplus c_{i_c,j}$ to recover the remaining data sectors. This process ultimately reconstructs the complete original file.

5) **Data Update**: When desiring to update the outsourced data, U submits a dynamic request to CSP.

- *Inserting Data:* When inserting a new data block m^* after the i-th position, U first processes the block by either: (a) for confidential data, computing sector keys $k_j^* = H_1(m_j^*)$ and ciphertext of sectors $c_j^* = H_3(k_j^*) \oplus m_j^*$ to form c^*, or (b) for public data, directly setting $c^* = m^*$; then U generates the block tag $tg^* = H_1(c^*)$ and sends the initial insertion request $\langle \text{Insert}, UID, W, t, i, tg^* \rangle$ to the CSP, which authenticates U by verifying $e(UID, g) \stackrel{?}{=} e(H_4(uid) \cdot t, W)$ and performs duplicate checking. If the block is unique, U computes $y^* = (\sum_{j=1}^{s} k_j^*) \oplus \gamma$, the public value $Y^* = g^{y^*}$, and the authenticator $\sigma^* = (H_2(s\|tg^*) \cdot \prod_{j=1}^{s} r_j^{c_j^*})^{y^*}$ before submitting the final insertion request $\langle \text{Insert}, c^*, \sigma^* \rangle$. Upon receiving the final insertion request, the CSP performs the following operations: (a) stores the new data block; (b) invokes the Insert algorithm to update the MHT by inserting a new leaf node after position i and recomputing the hash values, thereby obtaining the new root w_r^* (as illustrated in Fig. 2); and (c) uploads the update tuple $\langle t^*, tg^*, \sigma^*, uid, SID, w_r^* \rangle$ to the blockchain, where the smart contract's verification of w_r^* confirms that the CSP has properly executed the data update.

Similarly, when U needs to perform batch insertion of data blocks $\{m_j^*\}_{1 \leq j \leq \eta}$, where η is the quantity of data blocks inserted in batches, it only encrypts the private data blocks to obtain the processed confidential data blocks and public

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on March 29,2026 at 13:35:55 UTC from IEEE Xplore. Restrictions apply.

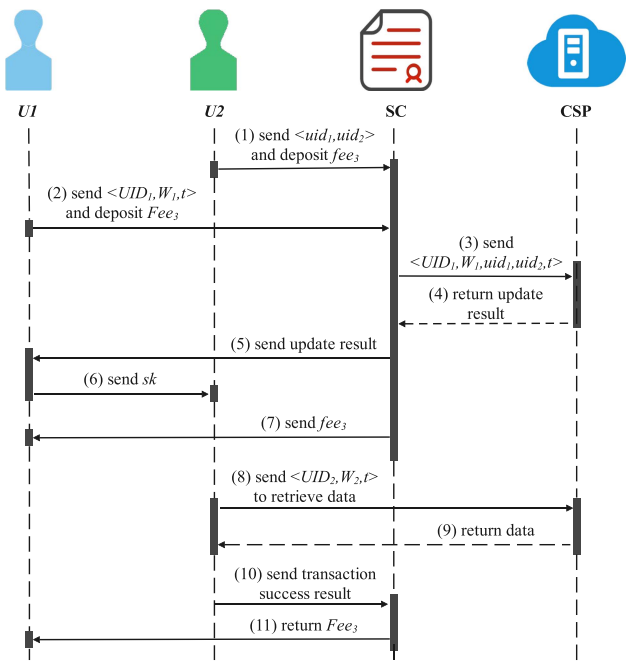


Fig. 3. The process of data ownership transaction.

data block set $\{c_j^*\}$. After computing the corresponding data block tags $\{tg_j^*\}$, U submits the batch insertion request to the CSP. Following successful authentication and deduplication verification, the CSP returns the processing results to U. Upon receiving the authenticator set $\{\sigma_j^*\}$ for the non-duplicate data blocks, the CSP: (1) stores all data blocks, (2) executes the Insert algorithm to perform batch insertion by updating the MHT, and (3) ultimately records the transaction with the updated root w_r^* and relevant metadata $\langle t^*, \{tg_j^*\}, \{\sigma_j^*\}, uid, SID, w_r^*\rangle$ to the blockchain, where the smart contract automatically verifies the integrity of w_r^* to ensure proper execution of the data update operation.

The procedures for data modification and deletion follow a similar protocol to the insertion process described above. However, to avoid redundancy, we omit the details.

6) **Data Ownership Transaction**: The ownership transfer protocol between $U1$ and $U2$ for cloud storage files is illustrated in Fig. 3, where SC denotes the smart contract deployed on the blockchain. $U2$ initiates the process by submitting an ownership replacement request containing both parties' identity tuples $\langle uid_1, uid_2\rangle$ (where uid_1 and uid_2 represent $U1$'s and $U2$'s identity IDs respectively), and depositing the data purchase fee fee_3 to the blockchain smart contract. Subsequently, $U1$ provides its authentication tuple $\langle UID_1, W_1, t\rangle$ and deposits a security margin $Fee_3 = 2fee_3$ to the smart contract, which then forwards the complete ownership update package $\langle UID_1, W_1, t, uid_1, uid_2\rangle$ to the CSP. The CSP performs identity verification checking $e(UID_1, g) \stackrel{?}{=} e(H_4(uid_1)\cdot t, W_1)$. Upon successful authentication, CSP updates the ownership records, returns the tuple $\langle t, T_u, uid_2, SID, w_r\rangle$ to the smart contract, and triggers the blockchain to create new blocks recording this outsourced data storage transaction.

Following the ownership update, $U1$ securely transmits the decryption key sk to $U2$ through a secret channel upon receiving confirmation from the smart contract. Simultaneously, the SC transfers the data transaction fee fee_3 paid by $U2$ to $U1$'s account. $U2$ can submit a data access request $\langle UID_2, W_2, t\rangle$ to the CSP, where $UID_2 = (H_4(uid_2)\cdot t)^{\mu_2}$ and $W_2 = g^{\mu_2}$, with μ_2 being a random number in $\mathbb{Z}_p^*$. The CSP verifies $U2$'s identity against the updated data ownership records by checking the equation: $e(UID_2, g) \stackrel{?}{=} e(H_4(uid_2)\cdot t, W_2)$. Upon retrieving the encrypted data $\langle \mathbb{C}, Key_i\rangle$ from the cloud, $U2$ attempts decryption using the secret key sk. Successful decryption confirms $U1$'s honest transfer of the correct decryption key to $U2$, prompting $U2$ to transmit a transaction success notification to the SC, which subsequently refunds the security deposit Fee_3 to $U1$. If the decryption fails, it indicates that $U1$ has transmitted an incorrect decryption key. In this case, $U2$ will send a transaction failure notification to the SC. In response to this dishonest conduct, the SC automatically transfers $U1$'s deposited Fee_3 to $U2$'s account as compensatory damages for the failed transaction. Notably, since all authenticators and data tags are computed solely from the data content itself, the CSP requires no modifications to these security parameters after the transaction. This design ensures that $U2$ can successfully verify data integrity and access the outsourced content, while $U1$ permanently loses access privileges after the transaction, thereby achieving secure access control.

V. Security Analysis

In this section, we analyze the security of the proposal based on the defined security objectives.

Theorem 1: The audit evidence P generated by the CSP can be validated by the audit of the smart contract if all entities have honestly implemented the audit program.

Proof: The smart contract utilizes the (8) to verify the correctness of the audit evidence through the following steps:

$$
\begin{aligned}
&e(\sigma_c, g)\\
&= e\left(\prod_{i=1}^{z}\sigma_{x_i}^{v_i}, g\right)\\
&= e\left(\prod_{i=1}^{z}\left(H_2(s\|tg_{x_i})\cdot\prod_{j=1}^{s}r_j^{c_{x_i,j}}\right)^{y_{xi}\cdot v_i}, g\right)\\
&= e\left(\prod_{i=1}^{z}\left(H_2(s\|tg_{x_i})\cdot\prod_{j=1}^{s}r_j^{c_{x_i,j}}\right)^{v_i}, g^{\sum_{i=1}^{z}y_{x_i}}\right)\\
&= e\left(\prod_{i=1}^{z}\left(H_2(s\|tg_{x_i})^{v_i}\cdot\prod_{j=1}^{s}r_j^{v_i c_{x_i,j}}\right), \prod_{i=1}^{z}Y_{x_i}\right)\\
&= e\left(\prod_{i=1}^{z}H_2(s\|tg_{x_i})^{v_i}\cdot\prod_{j=1}^{s}r_j^{\sum_{i=1}^{z}v_i c_{x_i,j}}, \prod_{i=1}^{z}Y_{x_i}\right)
\end{aligned}
$$

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on March 29,2026 at 13:35:55 UTC from IEEE Xplore. Restrictions apply.

$$= e\left(\prod_{i=1}^{z} H_2(s\|tg_{x_i})^{v_i} \cdot \prod_{j=1}^{s} r_j^{P_j}, \prod_{i=1}^{z} Y_{x_i}\right). \tag{9}$$

Theorem 2: Based on the Computational Diffie-Hellman (CDH) assumption, no PPT adversary can pass the authenticator validation with non-negligible probability during auditing stage. Thus, our scheme satisfies authenticator unforgeability.

Proof: The theorem will be proved in a series of games between the challenger $\mathcal{C}$ and adversary $\mathcal{A}$.

Game 0. If $\mathcal{A}$ submits an authenticator σ_c' that does not match the expectation, i.e. $\sigma_c' \neq \sigma_c$, then $\mathcal{C}$ rejects it and terminates the game.

Assuming that $\mathcal{A}$ can win *Game 0* with non-negligible probability $\epsilon > negl(\lambda)$ within time t on a given instance, $(\mathbb{G}, g, g^{\sum_{i=1}^{n} y_i}, h)$, where $g, h \in \mathbb{G}$, we can construct a simulator $\mathcal{S}$, which, after at most q_h H_2 queries and q_a authenticator queries, is able to solve the CDH problem by computing $h^{\sum_{i=1}^{n} y_i}$ with a non-negligible probability $\epsilon' \geq \epsilon/(q_a \cdot 2e)$ in time $t' \leq t + O(q_h + q_a)$. $\mathcal{S}$ interacts with $\mathcal{A}$ in a simulation as follows:

- *System setup:* Assuming the outsourced file with n blocks $\{c_i\}_{1\leq i\leq n}$, where $c_i = \{c_{i,j}\}_{1\leq i\leq n, 1\leq j\leq s}$. $\mathcal{S}$ chooses random numbers $\{\alpha_j\}_{1\leq j\leq s}, \{\beta_j\}_{1\leq j\leq s}, \{v_i\}_{1\leq i\leq n} \in \mathbb{Z}_p^*$ and computes $r_j = g^{\alpha_j} h^{\beta_j}$.
- *Query:* $\mathcal{A}$ initiates a query to obtain the data block hash and the authenticator.
- *Authenticator forgery:* $\mathcal{S}$ constructs a random oracle H_2, maintaining a mapping table for each query such that $H_2(s\|tg_i) \leftrightarrow g^{v_i}$. If the query does not exist in the table, $\mathcal{S}$ computes $H_2(s\|tg_i)$ and tosses a coin $\tau \in \{0, 1\}$, where the probability of $\tau = 0$ is ϕ, and the probability of $\tau = 1$ is $1 - \phi$. $\mathcal{S}$ programs the randomized prediction machine according to $H_2(s\|tg_i) = g^{v_i} / \prod_{j=1}^{s} r_j^{c_{i,j}}$. Next, $\mathcal{A}$ can forge the authenticator:

$$\begin{aligned}\sigma_i &= \left(H_2(s\|tg_i) \cdot \prod_{j=1}^{s} r_j^{c_{i,j}}\right)^{y_i} \\ &= \left(g^{v_i} / \prod_{j=1}^{s} r_j^{c_{i,j}} \cdot \prod_{j=1}^{s} r_j^{c_{i,j}}\right)^{y_i} \\ &= (g^{v_i})^{y_i}. \end{aligned} \tag{10}$$

- *CDH problem solved:* When the conditions of *Game 0* hold, $\mathcal{A}$ passes the verification but the aggregation authenticator $\sigma_c' \neq \sigma_c$. Letting $\Delta P_j = P_j' - P_j = \sum_{i=1}^{n} v_i c_{i,j}' - \sum_{i=1}^{n} v_i c_{i,j} = \sum_{i=1}^{n} v_i (c_{i,j}' - c_{i,j})$, we can further obtain the following:

$$\begin{aligned} e\left(\frac{\sigma_c'}{\sigma_c}, g\right) &= e\left(\prod_{i=1}^{n} \sigma_i'^{v_i} / \prod_{i=1}^{n} \sigma_i^{v_i}, g\right) \\ &= e\left(\frac{\prod_{i=1}^{n}\left(H_2(s\|tg_i) \cdot \prod_{j=1}^{s} r_j^{c_{i,j}'}\right)^{y_i v_i}}{\prod_{i=1}^{n}\left(H_2(s\|tg_i) \cdot \prod_{j=1}^{s} r_j^{c_{i,j}}\right)^{y_i v_i}}, g\right) \\ &= e\left(\frac{\prod_{i=1}^{n}\left(\prod_{j=1}^{s} r_j^{c_{i,j}'}\right)^{v_i}}{\prod_{i=1}^{n}\left(\prod_{j=1}^{s} r_j^{c_{i,j}}\right)^{v_i}}, g^{\sum_{i=1}^{n} y_i}\right) \\ &= e\left(\frac{\prod_{j=1}^{s} r_j^{\sum_{i=1}^{n} v_i c_{i,j}'}}{\prod_{j=1}^{s} r_j^{\sum_{i=1}^{n} v_i c_{i,j}}}, \prod_{i=1}^{n} Y_i\right) \\ &= e\left(\prod_{j=1}^{s} r_j^{\Delta P_j}, \prod_{i=1}^{n} Y_i\right) \\ &= e\left(\prod_{j=1}^{s} g^{\alpha_j \Delta P_j} h^{\beta_j \Delta P_j}, \prod_{i=1}^{n} Y_i\right). \end{aligned} \tag{11}$$

Next, converting (11) yields (12):

$$\begin{aligned} e\left(\frac{\sigma_c'}{\sigma_c \cdot \prod_{i=1}^{n}\prod_{j=1}^{s} Y_i^{\alpha_j \Delta P_j}}, g\right) \\ = e\left(\prod_{j=1}^{s} h^{\beta_j \Delta P_j}, \prod_{i=1}^{n} Y_i\right). \end{aligned} \tag{12}$$

Now we can get the expected output of the simulator $\mathcal{S}$:

$$h^{\sum_{i=1}^{n} y_i} = \left(\frac{\sigma_c'}{\sigma_c \cdot \prod_{i=1}^{n}\prod_{j=1}^{s} Y_i^{\alpha_j \Delta P_j}}\right)^{\frac{1}{\sum_{j=1}^{s} \beta_j \Delta P_j}}. \tag{13}$$

Eventually, after performing q_h H_2 queries and q_a authenticator queries, the adversary $\mathcal{A}$ can win the game with advantage ϵ within time t. The probability that the simulator $\mathcal{S}$ successfully interacts with $\mathcal{A}$ exceeds $(1-\phi)^{q_a}$, and thus the simulator $\mathcal{S}$ achieves a solution $h^{\sum_{i=1}^{n} y_i}$ to the CDH problem with probability $\epsilon' \geq \epsilon\phi(1-\phi)^{q_a} \geq \epsilon/(q_a \cdot 2e)$ within time $t' \leq t + O(q_h + q_a)$. There exists a PPT algorithm $\mathcal{S}$ that solves the CDH problem if the adversary $\mathcal{A}$ is able to construct legal authenticators for the data blocks. Since solving the CDH problem is computationally infeasible, the probability that $\mathcal{A}$ wins the game is negligible. Therefore, our scheme satisfies authenticator unforgeability.

Theorem 3: Based on the Discrete Logarithm (DL) assumption, no PPT adversary can pass the proof validation with non-negligible probability in the data auditing stage, our scheme satisfies proof unforgeability.

Proof: Similarly, we construct a game to demonstrate the proof of this theorem.

Game 1. If $\mathcal{A}$ passes authenticator verification but submits audit evidence that does not match expectations, $\mathcal{C}$ rejects it and terminates the game.

Assuming that $\mathcal{A}$ can win *Game 1* with non-negligible probability ϵ, we can construct a simulator $\mathcal{S}$. Given the instances $(\mathbb{G}, g, h = g^a)$, we can solve the DL problem, computed to obtain a. The behavior of $\mathcal{S}$ is as follows:

- *System setup:* The notations involved in this game are consistent with *Game 1*. The file consists of n data blocks

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on March 29,2026 at 13:35:55 UTC from IEEE Xplore. Restrictions apply.

$\{c_i\}_{1\le i\le n}$, where $c_i = \{c_{i,j}\}_{1\le i\le n, 1\le j\le s}$. $\mathcal{S}$ chooses random numbers $\{\alpha_j\}_{1\le j\le s}, \{\beta_j\}_{1\le j\le s}, \{v_i\}_{1\le i\le n} \in \mathbb{Z}_p^*$ and computes $r_j = g^{\alpha_j} h^{\beta_j}$.

- *DL problem solved:* When the conditions of *Game 1* hold, $\mathcal{A}$ passes the authenticator verification $\sigma_c' = \sigma_c$, but the evidence is not equal $P' \neq P$ and can be obtained:

$$e\left(\frac{\sigma_c'}{\sigma_c}, g\right) = e\left(\prod_{j=1}^{s} g^{\alpha_j \Delta P_j} h^{\beta_j \Delta P_j}, \prod_{i=1}^{n} Y_i\right). \quad (14)$$

Due to $\sigma_c' = \sigma_c$, it is possible to further compute:

$$\prod_{j=1}^{s} g^{\alpha_j \Delta P_j} h^{\beta_j \Delta P_j} = 1. \quad (15)$$

Now it can be calculated:

$$h = g^{-\frac{\sum_{j=1}^{s} \alpha_j \Delta P_j}{\sum_{j=1}^{s} \beta_j \Delta P_j}}. \quad (16)$$

Finally, since $P' \neq P$, the probability that $\sum_{j=1}^{s} \beta_j \Delta P_j = 0$ is $1/q$, and thus $\mathcal{S}$ can compute a solution to the DL problem with non-negligible probability $1 - 1/q$. Thus, if there exists a PPT adversary $\mathcal{A}$ who wins in *Game 1* with non-negligible probability, a solution to DL can be constructed, contradicting the defined assumptions. Consequently, our scheme satisfies the proof unforgeability.

Theorem 4: The proposed scheme protects the privacy of the outsourced data and no attacker and malicious CSP can obtain the plaintext based on the encrypted data.

Proof: In our scheme, U encrypts the confidential data before uploading the outsourced data. By generating the encryption key $k_{i,j} = H_1(m_{i,j})$, where $1 \le i \le n, 1 \le j \le s$, U encrypts the confidential data sector utilizing $c_{i_c,j} = H_3(k_{i_c,j}) \oplus m_{i_c,j}$, where $i_c \in I_2$. No attacker can recover the cipher data without knowing the encryption key $k_{i,j}$. Thus our scheme prevents malicious CSPs and attackers from recovering the plaintext even if they illegally obtain the encrypted data, ensuring the privacy of the data.

Theorem 5: The proposed scheme achieves proof of identity verification that data will not be illegally accessed and acquired by any unauthorised entity.

Proof: CSP stores the owner identity uid of the outsourced data and conducts identity authentication by $e(UID, g) = e((H_4(uid) \cdot t)^{\mu}, g) = e(H_4(uid) \cdot t, W)$. The outsourced data is not illegally accessed and obtained by any unauthorised entity through the authentication mechanism. Once the original owner initiates the ownership transaction, CSP updates the stored user identity uid to prevent the original data owner from illegally accessing the data.

Theorem 6: Our scheme achieves secure and correct transfer of data from the original data owner to the data purchaser.

Proof: In our scheme, the original data owner (denoted as $U1$) and the data purchaser (denoted as $U2$) execute the transaction through SC. $U2$ initiates the process by submitting an ownership transfer request $\langle uid_1, uid_2 \rangle$, which uniquely identifies both parties and ensures that the transaction is restricted to $U1$ and $U2$. Subsequently, $U1$ provides identity verification information $\langle UID_1, W_1, t \rangle$ to confirm data ownership, thereby preventing unauthorized access or fraudulent transactions. After authenticating the owner's identity, the CSP updates the ownership record based on the validated transaction details, ensuring the correctness of the ownership transfer.

To mitigate potential dishonesty by $U1$, a security deposit $Fee_3 = 2fee_3$ must be submitted to the SC as a guarantee of $U1$'s commitment to the transaction. If $U1$ engages in malicious behavior during the process, the SC automatically transfers the deposit to $U2$ as compensation, thereby safeguarding $U2$'s rights. If $U2$ is unable to decrypt the data after receiving it along with the decryption key sk, a transaction failure notification is sent to the SC, serving as evidence of $U1$'s misconduct. In this case, the SC transfers the deposit to $U2$, while $U1$ incurs a financial penalty. Given the assumption that rational parties will not act against their own interests, this mechanism effectively deters dishonest behavior by $U1$ and ensures the security of data transactions.

VI. Evaluation

In this section, we evaluate the proposed scheme in terms of theoretical analysis and simulation results.

A. Theoretical Analysis

Through the analysis of relevant blockchain-based schemes, we compare our approach with state-of-the-art schemes, including those proposed by Zhang et al. [9], Tian et al. [35], Liu et al. [34], and Zhang et al. [10], in terms of computational, communication, and storage overhead. These schemes share similar characteristics or objectives with ours, as they all employ blockchain technology and simultaneously achieve data deduplication and data auditing. Conducting theoretical analysis and subsequent simulation-based comparisons between our scheme and these methods can more comprehensively demonstrate the advantages and significance of the proposed scheme. Since Zhang et al.'s scheme [10] supports only file-level deduplication, we restrict the overhead comparison to the other three schemes, which support block-level deduplication. The results are summarized in Tables III—IV. For [10], we specifically compare it with the file-level deduplication component of our proposed scheme, with detailed results presented in the simulation experiments subsection. It is also important to note that although [9] adopts a multi-copy storage model, we assume a single-copy storage setting to ensure fairness. Moreover, all hash operations are uniformly denoted as H, as performance differences among different hash functions are negligible.

Let H, $ZMul$, $ZAdd$, Mul, Exp, $Pair$, Xor, Enc, and Dec denote the computational costs of hash operations, multiplication and addition in Z_p^*, multiplication and exponentiation in $\mathbb{G}$, bilinear pairing, XOR, and HCE2 encryption/decryption (as defined in [40]), respectively. We define l, $|F|$, $|Z_p^*|$, $|\mathbb{G}|$, $|T|$, $|L|$, $|M|$, and $|SD|$ as the lengths of strings, file F, Z_p^* elements, $\mathbb{G}$ elements, data index table, storage node list, data reconstruction matrix, and ciphertext tree. Finally, D denotes the ratio of non-duplicate data within file F.

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on March 29,2026 at 13:35:55 UTC from IEEE Xplore. Restrictions apply.

TABLE III
COMPARISONS OF COMPUTATION COSTS

Scheme	Data upload		Data audit		Data encrypt/decrypt	
	Key generation	Tag and authenticator generation	Prove	Verify	Encrypt	Decrypt
Zhang *et al.* [9]	nH	$(2n+1)H + nsMul + n(s+1)Exp$	$szZMul + s(z-1)ZAdd + zH$	$zH + (2z+s)Exp + (2z+s-2)Mul + 2Pair$	$nEnc$	$nDec$
Tian *et al.* [35]	$(N+1)H$	$NMul + (2N+1)Exp + (2N+1)H$	$z_1(ZMul + H) + (z_1-1)ZAdd$	$z_1H + (2z_1+1)Exp + (2z_1-1)Mul + 2Pair$	$NEnc$	$NDec$
Liu *et al.* [34]	$N(4H+2Exp)$	$N(3Exp + 4H + 2Mul + 2ZMul + ZAdd + Enc)$	$z_1Exp + (z_1-1)Mul + z_1ZMul + (z_1-1)ZAdd$	$(z_1+1)H + Exp + Mul + 2Pair + (z_1-1)ZAdd + 3z_1ZMul$	$NEnc$	$NDec$
Our scheme	$(ns+1)H$	$(ds+2d+1)Exp + (d+n)H + d((s-1)ZAdd + Xor + Mul)$	$zsZMul + s(z-1)ZAdd$	$zH + (2z+s)Exp + (3z+s-3)Mul + 2Pair$	$\omega ns(H+Xor)$	

TABLE IV
COMPARISONS OF COMMUNICATION AND STORAGE COSTS

Scheme	Communication costs				Storage costs		
	Upload	Audit	Update	Retrieve	CSP	U	Blockchain
Zhang *et al.* [9]	$n(\lvert Z_p^*\rvert + \lvert\mathbb{G}\rvert) + D\lvert F\rvert$	$(s+2)\lvert Z_p^*\rvert + \lvert\mathbb{G}\rvert$	–	$\lvert F\rvert$	–	$(n+2)\lvert Z_p^*\rvert$	$3\lvert Z_p^*\rvert + \lvert\mathbb{G}\rvert + \lvert T\rvert + \lvert L\rvert$
Tian *et al.* [35]	$(N+1)(\lvert Z_p^*\rvert + \lvert\mathbb{G}\rvert) + D\lvert F\rvert$	$(z_1+4)\lvert Z_p^*\rvert$	–	$\lvert F\rvert$	$2\lvert Z_p^*\rvert + N\lvert\mathbb{G}\rvert + D\lvert F\rvert$	$2\lvert Z_p^*\rvert + \lvert\mathbb{G}\rvert$	$3\lvert Z_p^*\rvert + 2\lvert\mathbb{G}\rvert + 2l$
Liu *et al.* [34]	$2\lvert Z_p^*\rvert + 3N\lvert\mathbb{G}\rvert + D\lvert F\rvert$	$6\lvert Z_p^*\rvert + \lvert\mathbb{G}\rvert$	–	$\lvert F\rvert$	$(DN+3)\lvert Z_p^*\rvert + (N+1)\lvert\mathbb{G}\rvert + D\lvert F\rvert + \lvert M\rvert + \lvert SD\rvert$	$\lvert Z_p^*\rvert$	$(N+10)\lvert Z_p^*\rvert + \lvert\mathbb{G}\rvert$
Our scheme	$n\lvert Z_p^*\rvert + (n+d+3)\lvert\mathbb{G}\rvert + l + D\lvert F\rvert$	$(s+3)\lvert Z_p^*\rvert$	$(s+2)\lvert Z_p^*\rvert + 6\lvert\mathbb{G}\rvert$	$\lvert F\rvert$	$(n+1)\lvert Z_p^*\rvert + (n+d+1)\lvert\mathbb{G}\rvert + l + D\lvert F\rvert$	$\lvert Z_p^*\rvert + \lvert\mathbb{G}\rvert$	$6\lvert Z_p^*\rvert + 2d\lvert\mathbb{G}\rvert + 2l$

In the data uploading phase, Tian et al. [35] and Liu et al. [34] divide files into N blocks with separate tag calculations. Our scheme and Zhang et al.'s scheme [9] partition data into n blocks, each containing s sectors ($N = ns$), where sector size matches scheme [35]'s block size. This allows computing only n tags, significantly reducing computational overhead. While schemes [9], [34], [35] generate encryption keys per block, our scheme creates sector-level keys for subsequent privacy protection and authenticator generation. During authenticator generation, we compute only d authenticators for deduplicated blocks, compared to full block authenticators in rest schemes, yielding lower computational overhead. Our scheme requires user identity verification before deduplication checking, introducing minimal communication overhead. By storing encryption keys at CSP, we increase some communication overhead but reduce local storage compared to Zhang et al.'s scheme [9], maintaining overall efficiency advantages.

During the data auditing phase, the number of challenged data blocks is set to z. However, since our scheme and Zhang et al.'s scheme [9] partition the data blocks into sectors, the number of challenge data blocks in Tian et al.'s [35] and Liu et al.'s [34] schemes must be adjusted to z_1 to ensure a consistent amount of data is challenged, where $\frac{z_1}{z} = s$. In the scheme proposed by Liu et al. [34], the CSP computes the aggregated authenticator proof, whereas in our approach, this computation is performed by the smart contract. Consequently, our scheme has some advantages in the $Prove$ phase. During the $Verify$ process, our scheme needs to aggregate the parameter Y_i, thus incurring $ZMul$ overhead, but compared to scheme [35], our computational overhead is still lower. Our scheme and schemes [9], [34] are all audited by the blockchain smart contract. The difference is that Tian et al.'s scheme [35] has different cloud servers acting as auditors. The communication overhead between the four schemes varies due to the various audit proof calculation methods.

Furthermore, to protect data confidentiality, all the schemes encrypt the data through various encryption techniques. However, different from schemes [9], [34], [35], only part of the private data is required to be encrypted in our scheme, so we only need to encrypt $(n-\zeta)s$ data sectors. Unlike other schemes, our scheme utilizes hashing and exclusive OR operations to ensure data privacy and security. When the content of private data in the outsourced data is less, the computational advantage of our scheme is more salient. To further illustrate the superiority of our scheme, we define $\omega = \frac{n-\zeta}{n}$ as the percentage of private data in the outsourced file. It is clear that as ω decreases i.e., the lower the percentage of private data in the data, the lower the computational overhead of our scheme for encryption and decryption. Similarly, our scheme's communication cost is lower as a result of the utilization of data block partitioning. In the data retrieval stage, we only need to decrypt the private data blocks, and thus the computational overhead is significantly lower than that of schemes [34], [35]. At this point, the communication cost of each scheme is $|F|$.

Moreover, our scheme supports both dynamic data operations and ownership transfers. Each data block updated by a user incurs a computational overhead of $(s+2)H + 3Exp + Mul + Xor + (s-2)ZAdd$ and a communication overhead of $(s+2)|Z_p^*| + 6|\mathbb{G}|$. For data ownership transfers, the CSP only needs to verify and update the user's identity information in the cloud, without requiring the computation of any data tags or authenticators. Specifically, when the CSP verifies the identity of the original data owner $U1$, it incurs a computational cost of $2Pair + Mul$. Similarly, when the data purchaser $U2$ accesses the data, the CSP requires $2Pair + Mul$ in computational

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on March 29,2026 at 13:35:55 UTC from IEEE Xplore. Restrictions apply.

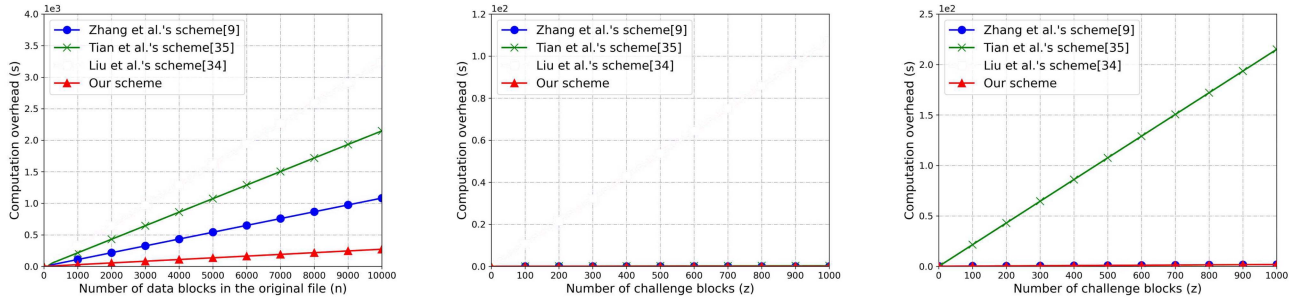


Fig. 4. Comparison of computation cost of upload and auditing. (a) Tag generation. (b) Prove. (c) Verify.

overhead to authenticate $U2$'s identity. During this process, $U1$ and $U2$ introduce communication overheads of $|Z_p^*| + 3|\mathbb{G}|$ and $2l + 3|\mathbb{G}|$ respectively, which remain within acceptable limits for practical implementations. Since the overhead of each data ownership transfer is minimal and independent of the number of data blocks, we have omitted further simulation evaluations of this aspect in subsequent analyses.

Lastly, in terms of storage overhead, the CSP in our scheme must maintain an improved MHT to support data dynamics in addition to storing the data files. By comparison, Liu et al.'s scheme [34] requires the CSP to store a data reconstruction matrix and a ciphertext tree to facilitate its deduplication mechanism. Regarding the storage overhead on the blockchain, Zhang et al.'s scheme [9] utilizes blockchain storage nodes for data storage, where the blockchain additionally maintains a data index table and a list of storage nodes. Tian et al. [35] employ the blockchain solely for record-keeping purposes. Similarly, the smart contracts in our scheme perform verification of audit proofs and data dynamics, which requires additional storage for authenticators and the MHT. This introduces further storage overhead; however, such overhead is considered acceptable.

B. Simulation Results

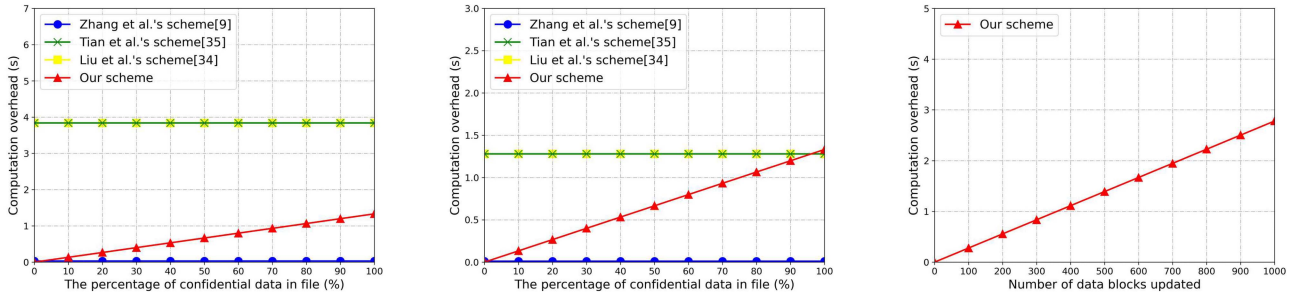
We conduct comparative experiments both off-chain and on-chain, respectively. In the on-chain experiments, our test environment is Ubuntu 24.04.1 LTS CPU 2, RAM 4 GB, ROM 20 GB with Intel(R) Core(TM) i7-10750H CPU @ 2.60 GHz (2.59 GHz) and 16 GB memory. In order to implement the HCE2 algorithm, we employ SHA-256 to generate the convergent key and AES-256 to encrypt the outsourced data. We use Python to call the Pairing Based Cryptography (PBC) Library and GNU Multiple Precision Arithmetic (GMP) Library to realize the scheme. For the on-chain implementations, we develop all schemes using Solidity and employ Ganache to simulate a private blockchain environment for testing.

1) Off-Chain Costs: Based on the findings of Ng et al. [41], $4KiB$ is determined as the optimal data block size for deduplication efficiency. Consequently, we set the data block size to $4KiB$, i.e., $\frac{|F|}{n} = 4KiB$, where each data block is divided into s sectors of the same size. We set $s = 128$ that is suitable for our scheme and experiment. The size of the data block in Tian et al.'s scheme [35] and Liu et al.'s scheme [34] is the same as the size of our data sectors. Zhang et al.'s scheme [9] has the same process for data blocks as ours. We set the degree of data duplication in the file to 75% in the experiment, and average the results 1000 times per experiment. We set the length of elements in the cyclic group to 512 bits, with an order of 160, meaning that the length of elements in the finite field Z_p^* is 160 bits.

Figs. 4 and 5 show the results of comparing the computational overhead of our proposal with schemes [9], [34], [35]. In the tag and authenticator generation phase, we vary the number of data blocks from 1000 to 10000. As shown in Fig. 4(a), our scheme demonstrates significantly lower computational overhead during the tag and authenticator generation phases compared to the other three schemes. While all schemes require computing data tags for all data blocks, both our scheme and Zhang et al.'s scheme [9] employ data partitioning, thus reducing computational costs during tag generation. Furthermore, unlike the other three schemes, our scheme only needs to compute authenticators for non-duplicate blocks after deduplication, whereas schemes [9], [34], [35] compute authenticators for all blocks prior to deduplication. This fundamental difference leads to substantially lower computational overhead in our authenticator generation phase compared to other schemes. Notably, the efficiency advantage of our scheme becomes more pronounced as the redundancy rate in the data increases.

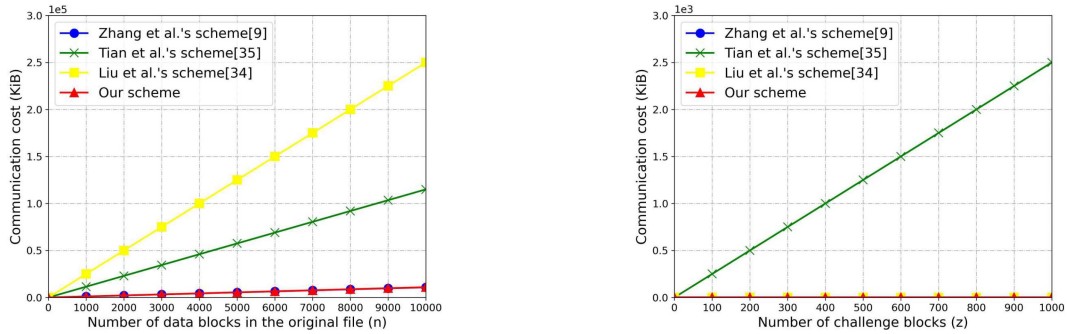
During the audit proof generation phase, as illustrated in Fig. 4(b), we vary the number of challenged data blocks from 100 to 1000. By partitioning the processing, our scheme achieves superior computational efficiency. For instance, with 300 and 460 challenged blocks, the computational overhead of Zhang et al.'s scheme [9] is 7.65 ms and 11.74 ms, respectively, while our scheme requires only 7.05 ms and 10.82 ms. These values are lower than those of Tian et al. [35] (83.87 ms and 128.59 ms) and markedly lower than those of Liu et al. [34]. The higher overhead in Liu et al.'s scheme stems from its computation of the aggregated authenticator proof at this stage, which incurs substantial computational cost. As shown in Fig. 4(c), we conduct audit evidence verification experiments for the number of challenge data blocks from 100 to 1000. Experimental results show that for challenged block counts of $z = 300$ and $z = 460$, the computational overhead of our scheme is 613.92ms and 883.36ms, respectively. The overhead of Zhang et al.'s

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on March 29,2026 at 13:35:55 UTC from IEEE Xplore. Restrictions apply.



(a) (b) (c)

Fig. 5. Comparison of computation cost of encrypt/decrypt and update. (a) Encrypt. (b) Decrypt. (c) Update.



(a) (b)

Fig. 6. Comparison of communication costs. (a) Upload. (b) Auditing.

scheme [9] is comparable to ours, measuring 612.72ms and 881.52ms, while Liu et al.'s approach [34] achieves significantly lower overhead at 93.34ms and 141.96ms. Although our scheme incurs marginally higher overhead compared to schemes [9], [34], it remains substantially more efficient than that of Tian et al.'s scheme [35], underscoring its notable advantages.

In the confidential data encryption phase, we examine the overhead spent on encrypting files with different confidential data content when the number of data blocks is 5000. Our scheme encrypts every private data sector for the security of private data and leaves the publicly available data untouched. In contrast, schemes [9], [34], [35] encrypt every data block. As shown in Fig. 5(a), since we are encrypting at a finer granularity using the sector key generated from the data itself, the computational overhead is slightly higher than Zhang et al.'s scheme [9] which encrypts for data blocks. Nevertheless, the encryption overhead of our scheme is significantly lower than that of Tian et al.'s scheme [35] and Liu et al.'s scheme [34] even when the percentage of confidential data in the outsourced file reaches 100% . Clearly, when the fraction of confidential data in the outsourced data is smaller, our scheme's computational cost is smaller and more practical. Similar to data encryption, the lower the fraction of confidential data in the outsourced data, the lower the computational overhead of our scheme when the data is decrypted. As shown in Fig. 5(b), only when the proportion of private data approaches 100% , does the computational overhead of our scheme become comparable to those of Tian et al.'s [35] and Liu et al.'s [34] schemes. Nevertheless, our scheme maintains the computational overhead of merely 1.33 s, which remains well within acceptable limits for practical deployment scenarios. Besides, although Zhang et al.'s scheme [9] has lower computational overhead, it has the local key management problem, which requires users to store the data key locally, which will incur additional storage overhead.

Contrary to the other three schemes, our scheme further implements data dynamics. In our experiments, we measure the computational overhead during dynamic data updates using user data insertion as a representative test case. As in Fig. 5(c), it also takes only 2.78 s when the user updates 1000 data blocks.

Fig. 6 demonstrates the comparison of communication costs between our scheme and schemes [9], [34], [35]. As shown in Fig. 6(a), the data upload communication overhead of Liu et al.'s scheme [34] is much larger than that of our scheme and Zhang et al.'s scheme [9]. This discrepancy primarily stems from the additional auxiliary information required for transmission in Liu et al.'s scheme [34]. In our scheme, to address the local key management problem, users are required to upload not only the deduplicated data and the authenticator but also the processed data key to the CSP. Therefore, the final communication overhead is comparable to that in scheme [9]. In the data auditing phase, as shown in Fig. 6(b), our scheme and scheme [9], [34] have much less communication overhead than Tian et al.'s

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on March 29,2026 at 13:35:55 UTC from IEEE Xplore. Restrictions apply.

TABLE V
COMPARISONS OF COMPUTATION COSTS FOR FILE-LEVEL DEDUPLICATION SCHEMES

Stage	Zhang *et al.* [10]	Our scheme
File tag generation	0.0008	0.0008
Prove	0.7890	0.0108
Verify	0.0022	0.8834
Encrypt	0.0081	0.2662
Decrypt	0.0023	0.2662

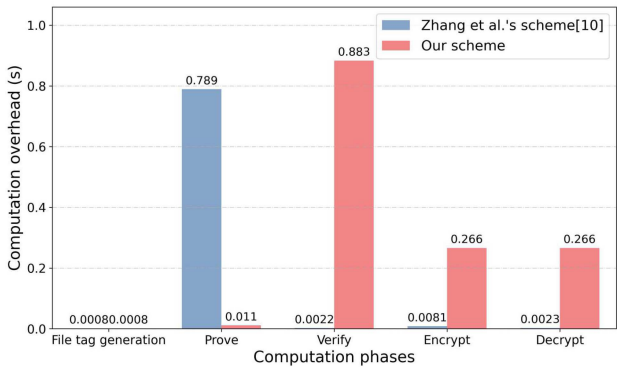


Fig. 7. Comparison of computation costs for file-level deduplication schemes.

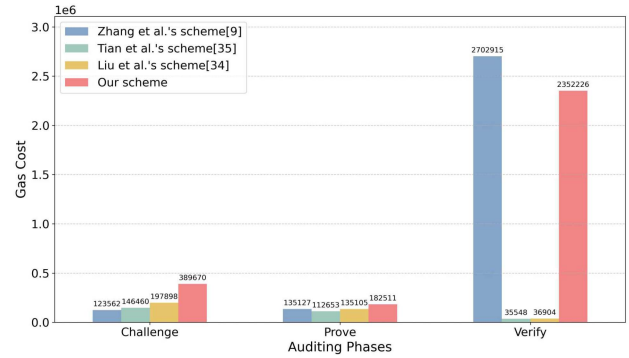


Fig. 8. Cost of gas for data auditing.

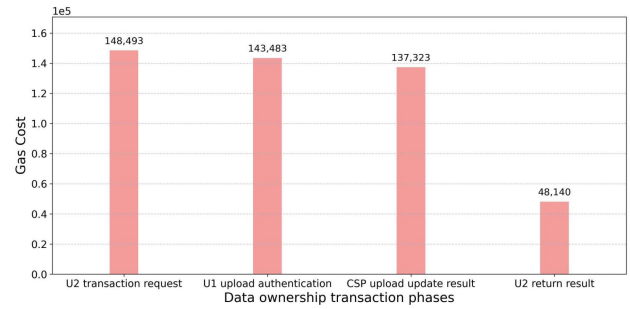


Fig. 9. Cost of gas for data ownership transactions.

scheme [35], because the auditing proof set size in our scheme and Zhang et al.'s scheme [9] is only related to the number of data block partitions s, and is independent of the number of challenged data blocks z. Meanwhile, the auditing proof in Liu et al.'s scheme [34] maintains a constant size regardless of these parameters.

Furthermore, since Zhang et al.'s scheme [10] supports only file-level deduplication, we conducted a specific comparative analysis, with results presented in Table V and Fig. 7. In our experiments, we set the file size to $2000 \times 4KiB$, with a privacy data ratio of $w = 50\%$ and 460 challenged data blocks during auditing. Both schemes require 0.0008s for file tag generation to support file-level deduplication. During auditing, Zhang et al.'s scheme [10] computes the aggregated authenticator proof in the $Prove$ stage, whereas our scheme performs it in the $Verify$ stage. Consequently, Zhang et al.'s scheme [10] incurs higher overhead in $Prove$, while ours exhibits slightly higher overhead in $Verify$. Regarding encryption, Zhang et al. use AES-256 for file encryption, while we encrypt individual privacy-sensitive data blocks. Although our approach incurs a marginally higher computational overhead ($\approx$ 0.2s difference), it significantly enhances security due to block-level encryption, with overhead remaining within acceptable practical limits.

2) On-Chain Costs: We compare the on-chain overhead incurred during the data auditing phase with the schemes [9], [34], [35]. The corresponding gas consumption results are presented in Fig. 8. In our scheme, payments for auditing fees and security deposits are required in the $Challenge$ and $Prove$ phases, resulting in moderately higher gas costs of 3.90×10^5 and 1.83×10^5, respectively, compared with other schemes. Both our method and those presented by Zhang et al. [9] and Liu et al. [34] employ smart contracts to perform integrity audit verification, necessitating computational operations. Consequently, the gas required in the $Verify$ phase is 2.35×10^6 in our scheme, whereas Zhang et al.'s scheme [9] consumes 2.70×10^6. Although our model incurs certain on-chain costs during the auditing stage, we introduce a penalty mechanism between users and the CSP that uses the smart contract to automatically determine audit results and penalize dishonest CSPs by deducting deposits. This design strengthens the security of the auditing process and mitigates the risk of CSP misbehavior.

Simultaneously, we deploy the smart contract for data ownership transactions within our proposed scheme, with the gas consumption at each stage illustrated in Fig. 9. When $U2$ initiates a data ownership transfer request, they are required to upload relevant user identity information and prepay the transaction fee, which incurs a gas cost of 1.48×10^5. Concurrently, $U1$ must also submit identity verification to prove ownership and prepay a deposit, consuming 1.43×10^5 gas. After the CSP verifies the user identities and updates the cloud data ownership information, it returns the update result to the smart contract, a process that consumes 1.37×10^5 gas. Finally, $U2$ returns the transaction result to the smart contract, which then refunds the corresponding deposit based on the outcome, incurring a gas cost of 0.48×10^5. The data ownership transaction process does not require extensive computation and only involves the transfer of fees, making the associated gas costs affordable for users.

VII. CONCLUSION

To address the challenges of outsourced data redundancy and confidential data security in cloud storage, we present an

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on March 29,2026 at 13:35:55 UTC from IEEE Xplore. Restrictions apply.

efficient data deduplication and mixed auditing scheme based on blockchain. Specifically, we employ data tagging to design a more efficient deduplication mechanism that supports data isolation and prevents users from incurring costs for duplicated content. To further ensure data security and integrity, we implement mixed spontaneous auditing of plaintext and ciphertext data through smart contracts, thereby enabling secure integrity verification without imposing computational overhead on users. Building on this foundation, our scheme simultaneously supports dynamic data operations, including insertion, modification, and deletion, as well as ownership transfers of outsourced data among users, thereby accommodating diverse user requirements. Theoretical analysis and experimental evaluations demonstrate that our proposal surpasses state-of-the-art approaches by reducing overhead in both tag generation and auditing, highlighting its feasibility and practical advantages.

Future work can further investigate auditing techniques for cloud storage to enhance compatibility, enabling applicability in broader contexts such as multi-cloud environments and distributed systems. Moreover, it is also interesting to consider strengthening the scheme's resilience against emerging threats, particularly advanced attack vectors, including quantum attacks.

References

[1] L. Zhou, A. Fu, S. Yu, M. Su, and B. Kuang, "Data integrity verification of the outsourced Big Data in the cloud environment: A survey," *J. Netw. Comput. Appl.*, vol. 122, pp. 1–15, 2018.

[2] F. Chen, J. Cai, T. Xiang, and X. Liao, "Practical cloud storage auditing using serverless computing," *Sci. China Inf. Sci.*, vol. 67, no. 3, 2024, Art. no. 132102.

[3] L. Wang, M. Hu, Z. Jia, Z. Guan, and Z. Chen, "SStore: An efficient and secure provable data auditing platform for cloud," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 4572–4584, 2024.

[4] W. Li, W. Susilo, C. Xia, L. Huang, F. Guo, and T. Wang, "Secure data integrity check based on verified public key encryption with equality test for multi-cloud storage," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 6, pp. 5359–5373, Nov./Dec. 2024.

[5] C. Zhang, H. Xuan, T. Wu, X. Liu, G. Yang, and L. Zhu, "Blockchain-based dynamic time-encapsulated data auditing for outsourcing storage," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 1979–1993, 2024.

[6] G. Tian, J. Wei, M. Miao, F. Guo, W. Susilo, and X. Chen, "Blockchain-based compact verifiable data streaming with self-auditing," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 4, pp. 3917–3930, Jul./Aug. 2024.

[7] Y. Yang, Y. Chen, F. Chen, and J. Chen, "Identity-based cloud storage auditing for data sharing with access control of sensitive information," *IEEE Internet Things J.*, vol. 9, no. 13, pp. 10434–10445, Jul. 2022.

[8] J. Gantz, and D. Reinsel, *The Digital Universe Decade-Are You Ready?*, Framingham, MA, USA, International Data Corporation, 2010,

[9] Q. Zhang, D. Sui, J. Cui, C. Gu, and H. Zhong, "Efficient integrity auditing mechanism with secure deduplication for blockchain storage," *IEEE Trans. Comput.*, vol. 72, no. 8, pp. 2365–2376, Aug. 2023.

[10] Q. Zhang, S. Qian, J. Cui, H. Zhong, F. Wang, and D. He, "Blockchain-based privacy-preserving deduplication and integrity auditing in cloud storage," *IEEE Trans. Comput.*, vol. 74, no. 5, pp. 1717–1729, May 2025.

[11] X. Yang, R. Lu, J. Shao, X. Tang, and A. A. Ghorbani, "Achieving efficient and privacy-preserving multi-domain Big Data deduplication in cloud," *IEEE Trans. Serv. Comput.*, vol. 14, no. 5, pp. 1292–1305, Sep./Oct. 2021.

[12] J. Li, Y. Li, J. Wu, Z. Zhang, and Y. Jin, "Blockchain-based decentralized cloud storage with reliable deduplication and storage balancing," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 4, pp. 3289–3304, Jul./Aug. 2024.

[13] R. Chen, Y. Mu, G. Yang, and F. Guo, "BL-MLE: Block-level message-locked encryption for secure large file deduplication," *IEEE Trans. Inf. Forensics Secur.*, vol. 10, no. 12, pp. 2643–2652, Dec. 2015.

[14] G. Ateniese et al., "Provable data possession at untrusted stores," in *Proc. 14th ACM Conf. Comput. Commun. Secur.*, 2007, pp. 598–609.

[15] Q. Wang, C. Wang, K. Ren, W. Lou, and J. Li, "Enabling public auditability and data dynamics for storage security in cloud computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 22, no. 5, pp. 847–859, May 2011.

[16] C. Liu et al., "Authorized public auditing of dynamic Big Data storage on cloud with efficient verifiable fine-grained updates," *IEEE Trans. Parallel Distrib. Syst.*, vol. 25, no. 9, pp. 2234–2244, Sep. 2014.

[17] H. Yan, J. Li, J. Han, and Y. Zhang, "A novel efficient remote data possession checking protocol in cloud storage," *IEEE Trans. Inf. Forensics Secur.*, vol. 12, no. 1, pp. 78–88, Jan. 2017.

[18] H. Yu et al., "Efficient dynamic multi-replica auditing for the cloud with geographic location," *Future Gener. Comput. Syst.*, vol. 125, pp. 285–298, 2021.

[19] J. Li, H. Yan, and Y. Zhang, "Efficient identity-based provable multi-copy data possession in multi-cloud storage," *IEEE Trans. Cloud Comput.*, vol. 10, no. 1, pp. 356–365, Jan.–Mar. 2022.

[20] R. Mishra, D. Ramesh, D. R. Edla, and L. Qi, "DS-chain: A secure and auditable multi-cloud assisted EHR storage model on efficient deletable blockchain," *J. Ind. Inf. Integration*, vol. 26, 2022, Art. no. 100315.

[21] J. Li, H. Yan, and Y. Zhang, "Identity-based privacy preserving remote data integrity checking for cloud storage," *IEEE Syst. J.*, vol. 15, no. 1, pp. 577–585, Mar. 2021.

[22] J. Li, H. Yan, and Y. Zhang, "Certificateless public integrity checking of group shared data on cloud storage," *IEEE Trans. Serv. Comput.*, vol. 14, no. 1, pp. 71–81, Jan./Feb. 2021.

[23] A. Fu, S. Yu, Y. Zhang, H. Wang, and C. Huang, "NPP: A new privacy-aware public auditing scheme for cloud data sharing with group users," *IEEE Trans. Big Data*, vol. 8, no. 1, pp. 14–24, Feb. 2022.

[24] C. Gai, W. Shen, M. Yang, and J. Yu, "PPADT: Privacy-preserving identity-based public auditing with efficient data transfer for cloud-based IoT data," *IEEE Internet Things J.*, vol. 10, no. 22, pp. 20065–20079, Nov. 2023.

[25] M. Wang, J. Yu, W. Shen, and R. Hao, "Privacy-preserving time-based auditing for secure cloud storage," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 7866–7878, 2024.

[26] X. Hu, J. Chang, T. Ahmad, F. Zhang, and Y. Zhang, "Identity-based integrity auditing scheme with sensitive information hiding for proxy-server-assisted cloud storage applications," *IEEE Internet Things J.*, vol. 12, no. 6, pp. 6673–6684, Mar. 2025.

[27] Y. Yang, Y. Chen, P. Xiong, F. Chen, and J. Chen, "Decentralized self-auditing multiple cloud storage in compressed provable data possession," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 4, pp. 3901–3915, Jul./Aug. 2025.

[28] Y. Lin, J. Li, S. Kimura, Y. Yang, Y. Ji, and Y. Cao, "Consortium blockchain-based public integrity verification in cloud storage for IoT," *IEEE Internet Things J.*, vol. 9, no. 5, pp. 3978–3987, Mar. 2022.

[29] Q. Zhang et al., "Efficient blockchain-based data integrity auditing for multi-copy in decentralized storage," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 12, pp. 3162–3173, Dec. 2023.

[30] Y. Miao, K. Gai, L. Zhu, K.-K. R. Choo, and J. Vaidya, "Blockchain-based shared data integrity auditing and deduplication," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 4, pp. 3688–3703, Jul./Aug. 2024.

[31] H. Yu, Y. Chen, Z. Yang, Y. Chen, and S. Yu, "EDCOMA: Enabling efficient double compressed auditing for blockchain-based decentralized storage," *IEEE Trans. Serv. Comput.*, vol. 17, no. 5, pp. 2273–2286, Sep./Oct. 2024.

[32] S. Wang, M. Yu, M. Zhao, and H. Wang, "Blockchain-based cloud data auditing scheme in multi-cloud storage service environment," in *Proc. 27th Int. Conf. Comput. Supported Cooperative Work Des.*, 2024, pp. 669–674.

[33] X. Zhang, Q. Liu, B. Liu, Y. Zhang, and J. Xue, "Dynamic certificateless outsourced data auditing mechanism supporting multi-ownership transfer via blockchain systems," *IEEE Trans. Netw. Service Manag.*, vol. 22, no. 2, pp. 2017–2030, Apr. 2025.

[34] B. Liu, X. Zhang, X. Yang, Y. Zhang, J. Xue, and R. Zhou, "Blockchain-assisted fine-grained deduplication and integrity auditing for outsourced large-scale data in cloud storage," *IEEE Internet Things J.*, vol. 12, no. 12, pp. 21662–21678, Jun. 2025.

[35] G. Tian et al., "Blockchain-based secure deduplication and shared auditing in decentralized storage," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 6, pp. 3941–3954, Nov./Dec. 2022.

[36] Y. Xu, C. Zhang, G. Wang, Z. Qin, and Q. Zeng, "A blockchain-enabled deduplicatable data auditing mechanism for network storage services," *IEEE Trans. Emerg. Topics Comput.*, vol. 9, no. 3, pp. 1421–1432, Jul.–Sep. 2021.

[37] R. Mishra, D. Ramesh, S. S. Kanhere, and D. R. Edla, "Enabling efficient deduplication and secure decentralized public auditing for cloud storage: A redactable blockchain approach," *ACM Trans. Manage. Inf. Syst.*, vol. 14, pp. 1–35, 2023.

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on March 29,2026 at 13:35:55 UTC from IEEE Xplore. Restrictions apply.

[38] M. Song, Z. Hua, Y. Zheng, H. Huang, and X. Jia, "Blockchain-based deduplication and integrity auditing over encrypted cloud storage," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 6, pp. 4928–4945, Nov./Dec. 2023.
[39] L. Peng, Z. Yan, X. Liang, and X. Yu, "Secdedup: Secure data deduplication with dynamic auditing in the cloud," *Inf. Sci.*, vol. 644, 2023, Art. no. 119279.
[40] M. Bellare, S. Keelveedhi, and T. Ristenpart, "Message-locked encryption and secure deduplication," in *Proc. Adv. Cryptol. - EUROCRYPT 2013*, 2013, pp. 296–312.
[41] C. Ng and P. P. C. Lee, "Revdedup: A reverse deduplication storage system optimized for reads to latest backups," in *Proc. Asia-Pacific Workshop Syst.*, 2013, pp. 1–7.

Chunfei Pan received the BS degree in computer science and engineering, Nanjing University of Science and Technology, in 2020. She is currently working toward the MS degree in software engineering with the School of Computer Science and Engineering, Nanjing University of Science and Technology. Her research interests include cloud storage and blockchain.

Lei Zhou received the BS degree in network engineering and the PhD degree in cyberspace security from the Nanjing University of Science and Technology, Nanjing, China, in 2015 and 2021, respectively. She is currently an associate professor with the School of Information Engineering, Suqian University, China. Her research interests include blockchain and cloud storage.

Anmin Fu (Member, IEEE) received the PhD degree in information security from Xidian University, Xi'an, China, in 2011. He is currently a professor of the Nanjing University of Science and Technology, China. He has authored or coauthored more than 100 technical papers, including international journals and conferences, such as I*IEEE Transactions on Dependable and Secure Computing*, *IEEE Transactions on Information Forensics and Security*, IEEE S&P, ACM CCS, NDSS, AsiaCCS, and ACISP. His research interests include IoT security, cloud computing security, and privacy preserving.

Zhenzhu Chen (Member, IEEE) received the PhD degree from the School of Computer Science and Engineering, Nanjing University of Science and Technology in 2022. She is currently a lecturer with the School of Computing and Artificial Intelligence, Southwestern University of Finance and Economics. Her research interests include artificial intelligence security and information security.

Huaqun Wang received the BS degree in mathematics education from the Shandong Normal University, in 1997, the MS degree in applied mathematics from the East China Normal University, China, 2000, and the PhD degree in information security from the Nanjing University of Posts and Telecommunications in 2006. He is currently a professor of the Nanjing University of Posts and Telecommunications, China. His research interests include applied cryptography, network security, and cloud computing security.

Yifeng Zheng (Member, IEEE) is currently an assistant professor with the Department of Electrical and Electronic Engineering, The Hong Kong Polytechnic University. His research works have been authored or coauthored in prestigious venues like USENIX Security, CCS, TIFS, and TDSC. He was an associate editor for *IEEE Transactions on Services Computing*. His current research interests include focused on machine learning security and privacy, secure networked systems, and secure outsourced computation. He was on the Technical Program Committee of many major international conferences, such as USENIX Security'26, EuroS&P'25, PETS'25, AsiaCCS'25, and ICDCS'24. He was the recipient of the Best Paper Award in the European Symposium on Research in Computer Security (ESORICS) 2021.

Yansong Gao (Senior Member, IEEE) received the MSc degree from the University of Electronic Science and Technology of China and the PhD degree from the University of Adelaide, Australia. He is currently a lecturer with the University of Western Australia. His current research interests include AI security and privacy, hardware security, and system security. He is also an associate editor for *IEEE Transactions on Information Forensics and Security* and *IEEE Transactions on Neural Networks and Learning Systems*.

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on March 29,2026 at 13:35:55 UTC from IEEE Xplore. Restrictions apply.